

Læringsutbyttebeskrivelser for TDT4160

Sist oppdatert 11/11-25

Overordnede læringsutbyttebeskrivelser

Kunnskaper:

- **K1:** Studenten skal kjenne til datamaskiners konstruksjon og virkemåte, inkludert hvordan man oppnår høy effektivitet.
- **K2:** Studenten skal forstå grensesnittet mellom programvare og maskinvare.
- **K3:** Studenten skal forstå hvordan man bygger enkle og effektive prosessorer, inkludert enkeltsykel, flersykel, og samlebåndsarkitekturer.
- **K4:** Studenten skal forstå prinsippene bak hvordan man bygger effektive minnesystemer, inkludert hurtigbuffer og virtuelt minne.
- **K5:** Studenten skal forstå hvordan abstraksjon og struktur benyttes for å oppnå høy effektivitet og til å håndtere kompleksitet i datamaskiner.

Ferdigheter:

- **F1:** Studenten skal være i stand til å formulere enkle programmer i assemblykode.
- **F2:** Studenten skal være i stand til å lese blokkdiagrammer.
- **F3:** Studenten skal kunne relatere blokkdiagrammer på ulike abstraksjonsnivå til hverandre.

Generell kompetanse:

- **G1:** Studenten skal forstå den generelle virkemåten til en datamaskin og kunne anvende denne kunnskapen i prosjekter på alle abstraksjonsnivå.

Merk: Læringsutbyttebeskrivelsene er ikke identisk med læringsutbyttebeskrivelsene på fagets offisielle nettside, men endringene er kun kosmetiske.

Spesifikke læringsutbyttebeskrivelser

Tema	Læringsutbytte	Oppgaver i boka
		Eksamens- oppgaver
T1.1: Datamaskin typer og de 7 store ideene	• Studenten kjenner til de 7 datamaskintypene og deres viktigste egenskaper. Ultralav effekt-systemer Innvevde systemer Mobiltelefoner Bærbare datamaskiner Stasjonære datamaskiner Datasentere Superdatamaskiner • Studenten kjenner til de 7 store ideene i datamaskinarkitektur. Pipelining, samlebånd. Ikke vente på at klærne tørker før man setter nye klede i oppvaskmaskinen. Overlapp operasjoner Eks: Samlebåndprosessor Abstraksjon, glem bort kompleksiteten Eks: Blokkdiagram, 1 og 0 i stedet for analoge verdier, osv. Common case fast, det som skjer ofte må gjøres raskere. Eks: ALU må være rask og effektiv fordi den brukes mye Parallellitet i tid: Utføre flere instruksjoner over tid, for eksempel ved pipelining. Eks: én person som gjør flere oppgaver samtidig, overlappende. Parallellitet i rom: Utføre flere instruksjoner samtidig på flere fysiske enheter. Eks. legg til flere ALUer, kjerner eller prosessorer. Flere parallelle datastier. Eksempel: SIMD, single instruction, multiple data, samme instruksjon brukes på flere data samtidig. Eks: flere personer som gjør oppgaver samtidig Prediksjon, gjetting av hva som kommer til å skje. Skal vi hoppe eller ikke Eks: Beq så gjetter datamaskinen at vi ikke hopper, som ofte er riktig.	• 1.1 • 1.2 <i>Ingen spesielt relevante¹</i>

	Minnehierarki. Smått, raskt minne for å lagre data som brukes mye og ofte, mens data som brukes sjeldnere kan ligge i et lavere, større minne som er tregere. Eks: Sram, raskest, Dram, cache, harddisk Pålitelighet gjennom redundans: Noe kommer til å gå galt i et stort og kompleks system. Derfor må noe annet ta over jobben og fikse ting dersom noe går galt eller ryker. Eks: dersom vi gjetter feil ved en prediksjon må vi ha programvare som rydder opp, venter eller annet. • Studenten kan beskrive størrelser med rett prefiks (for eksempel GB og GiB). GB: 10^9, GiB: 2^{30}	
T1.2: Under overflaten	• Studenten kjenner rollen til applikasjonsprogramvare og systemprogramvare, herunder operativsystemet og kompilatoren. Systemprogramvare: Styrer og administrerer dataaskinens programvare slik at andre programmer kan kjøre. Operativsystemet, drivere, systemvedlikehold, det som kobler sammen applikasjonsprogramvare og maskinvare Applikasjonsprogramvare: Koden, instruksjonene til et program. Kjør opp på systemprogramvaren Operativsystemet: Håndterer periferienheter (I/O), allokerer lagring og minne og beskytter applikasjonene fra hverandre slik at ting holdes separat Kompilator: Gjør kildekode om til maskinkode, assembly, så kan assembleren gjøre det om til 1 og 0 som datamaskinen forstår • Studenten kan beskrive de fem hovedkomponentene i en datamaskin. Inndata, data som kommer inn i datamaskinen for behandling. Tastatur, sensorer, mikrofon... Alt utenfor Utdata, data som sendes ut og til minnet. Output. Skjerm, høytaler, osv. Minne, der alle instruksjonene og dataen lagres Kontrollenhet, styrer og koordinerer hvordan datastien og resten av CPUen opererer. Tolker instruksjoner og sender kontrollsignaler til datasti og minne.	Ingen spesielt relevante

Datasti, stien dataen tar gjennom prosessoren. Delen av CPUen som håndterer dataflyten og utfører beregningene. Flytter data mellom registre, ALU, lagring osv.

- Studenten kan forklare prinsippet om lagrede program.

Minnet inneholder instruksjoner på samme måte som den inneholder data. Kan leses og utføres av CPU.

Instruction fetch: CPU henter instruksjoner fra minnet, en etter en.

Instruction decode: Dekodes av kontrollenhet

Execute: Fullfør operasjonen

- Studenten kan gjengi produksjonsprosessen for integrerte kretser.

Silicon ingot

Silicon wafer får UV-stråling på seg i et visst mønster som «tegner» kretsen. Skjer mange ganger på samme wafer for å få flere lag med krets og transistorer

Alle brikkene testes

Transport

Brikkene testes på nytt

Framme

T1.3: Ytelse	• Studenten kjenner til de viktigste ytelsesmetrikkene i datamaskinarkitektur. Klokkefrekvens: høyere frekvens, raskere utførelse av instruksjoner, men kan ikke være raskere enn tiden det tar å fullføre kritisk sti IPC - instruksjoner per klokkesykel. CPI - sykler per instruksjon. Disse avhenger av CPU-arkitektur. Pipelining, parallellitet osv. Gjennomstrømning (throughput): Mengden arbeid gjort per tid. Måles i MIPS (millioner instruksjoner per sek) eller FLOPS (floating point operations per sekund) Ytelse: beregnet etter tiden det tar å fullføre et program. $1/\text{kjøretid}$ Tid brukt på applikasjon (user time) og tid brukt på operativsystemet (system time) • Studenten forklare forskjellen på kjøretid og båndbredde og velge den mest hensiktsmessige for gitte arkitekturoppgaver. - Kjøretid: Tiden det tar å fullføre i et program, ofte i sekunder. Iron law - Båndbredde: Mengden data som overføres per tid. Eks. hvor mye data en cache kan levere til minne per sekund. Bruk kjøretid når du analyserer hele programmet eller totaltid Båndbredde når oppgaven handler om dataflyt, gjennomstrømning, f.eks når man bruker roofline-modellen. • Studenten kan forklare «The Iron Law» og kan bruke den til å forutsi hvordan endringer i arkitekturen påvirker kjøretid. Iron law: $\text{sek/program} = \text{instruks/program} * \text{sykler/instruks} * \text{sek/sykel}$ Bare se på formelen så kan man si hvordan noe påvirker kjøretid.	• Ord. 24: Oppg. 5.4
----------------------------	--	--

¹ «Ingen spesielt relevante» betyr at jeg ikke har funnet noen gamle eksamensoppgaver som jeg mener gir vesentlig trening i dette temaet. Det betyr ikke at det ikke eksisterer oppgaver som er innom temaet. For eksempel er noen av disse temaene er dekket av korte faktaspørsmål (typisk oppgave 1 i noen eksamener).

	• Studenten kjenner til hvordan spenning og klokkefrekvens påvirker effektforbruk og strømforbruk. Energi er proporsjonal med: $\frac{1}{2} \text{ kapasitans} * \text{spenning}^2$. En liten endring i ytelse kan potensielt øke effektforbruket betraktelig. Altså: effektforbruk er ikke lineært med ytelse. Høy ytelse -> enda høyere effektforbruk Høy klokkefrekvens: transistorene må skifte verdi ofte. Spenningen vil i praksis også øke, så en liten økning i klokkefrekvens kan gi større endring i effektforbruk. Høy sykel -> varmere prosessor -> trenger mer kjøling og får gjort mindre per energienhet. Overklokking: Superhøy frekvens, men bare så lenge temperaturen og energien tillater det. • Studenten vet hva en testprogramsamling er og hvorfor de brukes. Testprogramsamling: benchmark suite, samling standardiserte programmer eller tester som brukes for å måle, sammenligne og evaluere ytelsen til datamaskiner, prosessorer eller systemer. Hvert program i samlingen tester ulike ting, eks: Prosessorens regneevne Minnehastighet Parallell ytelse Filhåndtering Osv....		
T1.4: Parallele datamaskiner	• Studenten vet hvorfor stort sett alle datamaskiner i dag har mer enn én prosessorkjerne. Strøm / energibegrensning: Høy klokkehastighet fører til enda høyere effektforbruk. Møtte etter hvert en strømbegrensning. Man nådde den praktiske grensen for kjøling av kommersielle mikroprosessorer. Derfor avtok hastigheten for programmer på en enkelt prosessor. Energieffektivitet er også viktig. Flere prosessorkjerner - bedre ytelse per joule Større gjennomstrømning (throughput)	• 1.10 • 1.16 <i>Ingen spesielt relevante</i>	

	Øker parallelitet i rom • Studenten kan bruke Amdahl's lov til å analysere ytelsesforbedring i datamaskiner. Amdahls lov: Kjøretid etter forbedring = kjøretid påvirket av forbedring / hvor mye forbedret + kjøretid av det som ikke er forbedret Gevinsten av det du optimaliserer er alltid begrenset av det du ikke optimaliserer. Man må derfor måle først og så optimalisere etterpå. Eks: dersom man har flere registre, tar det lenger tid å aksessere dem. Mindre er rskere. Størst gevinst når man gjør det vanlige tilfellet raskt. Eks: forbedre multiplikasjon med 50%: Før: mul 20s, add 10s, fil 1s, annet 49s = 100s Etter: mul 10s, add 10s, fil 1s, annet 49s = 90s tid etter forbedring = 20s mul / 50% + 80s = 90s	
T2.1: Instruksjoner	• Studenten kjenner til de tre designprinsippene for instruksjonssettdesign og deres motivasjon. ISA, intruction set architecture, grensesnittet mellom programvare og maskinvare, språket til datamaskinen. I dette faget er RISC-V datamaskinens språk, ISA. En måte å gjemme bort detaljer for å håndtere kompleksitet (abstraksjon). Regelmessighet forenkler: Motivasjon: Holde maskinvaren enkel Opprettholde konsistens og forutsigbarhet i instruksjonsformatet. Hold maskinvaren enkel. Komplisert maskinvare kan få lenger kritisk sti, som senker klokkefrekvensen og ytelsen. Eks: Aritmetiske instruksjoner spesifiserer alltid 1 operasjon og 3 operander.	• 2.1 • 2.2 • 2.3 • 2.4 • 2.12 • 2.13 • Kont 25: Oppgave 1 • Ord. 24: Oppgave 1 • Kont 24: Oppgave 1 • Ord. 22: Oppg. 11²

Alle instruksjoner er av samme størrelse, og registerfeltene er på samme sted i alle instruksjonsformater (samme med opcode)

Mindre er raskere:

Motivasjon: hastighet er en kritisk faktor for ytelse.

Mindre antall eller mindre størrelse på maskinvaren fører ofte til raskere operasjoner. Lange datastier eller fysisk lange ledninger bruker mer tid på å sende gjennom data. I tillegg kan

Eks: 32 registre. Mange registre kan øke kompleksitet og kritisk sti, som senker klokkefrekvens og ytelse.

God design kverer gode kompromisser:

Motivasjon: Løser de uungåelige konfliktene som oppstår når man prøver å kombinere fordelene fra de 2 første prinsippene.

Når et designmål er i konflikt, må arkitektene velge et kompromiss som gir den beste totale løsningen

Eks: Holde alle instruksjoner like store (for enkelthet og regelmessighet) og ønske om å bruke store adresser eller konstanter. Kompromiss: Forskjellige typer instruksjonsformater: R, I, S, osv.

- Studenten kan oversette fra et høynivåspråk til assemblyinstruksjoner (og omvendt).

Bruk tabell, øv.

- Studenten kan oversette fra assemblyinstruksjoner til maskinkode (og omvendt).

Tabell og øving

- Studenten kjenner til instruksjonsformatene i RISC-V og kan forklare hvorfor de er definert slik de er.

R-type: aritmetikk. Rd - destinasjonsregister, rs1 og rs2 - første og andre operand.

Funct 3 og funct 7 - flere operasjonskoder, mer spesifikt enn opcode.

Shamt - skiftemengde

Opcode og funct3 er alltid på samme sted. Registerbitene er like i alle instruksjoner der de trengs.

Hvorfor:

Funct 3 og 7 gjør at man kan representere mange operasjoner med 1 opcode

Trenger 2 operander og 1 destinasjonsregister

I-type: Alle som bruker immediate gen. Addi, subi, lw (bruker en immediate for å finne skift), j

Hvorfor:

Sparer plass, mange operasjoner trenger ett register og en konstant.

Imm plasseres kompakt slik at dekode-logikken er enkel

Samme format kan brukes for ALU operasjoner og loads -> færre formattyper

S-type: sw, minneinstruksjoner som skriver et register til minne

Hvorfor:

Trenger 2 registre, ingen rd. Denne erstattes av imm. Imm deles i to for at funct og opcode ligger på faste posisjoner.

U-type: for store konstanter. Lui, auipc

	Hvorfor: gir stor imm B-type: branch-instruksjoner. Beq, bne, blt, Hvorfor: Sammenligner 2 registre, ingen rd. Bruker Sikkert greit • Studenten forstår hvordan instruksjoner lagres i minnet og hvordan dette utnyttes til å implementere kontrollflyt. Minnet: tabell av registerverdier Eks. 32 bits ord, altså 8 bytes. Hver byte er en «kolonne» i tabellen. Skift på 1 i minneinstruksjon: skift med 1 byte. Skift på 4: hopp til neste minneverdi Hver instruksjon ligger etter den forrige. Lokalitet i rom. CPU bruker program counter som peker på adressen til neste instruksjon. For hopp brukes PC-relative offset i branch og jump-instruksjoner. Programmene blir posisjonsuavhengige og kan dermed flyttes rundt i minnet. Koden blir mer kompakt og dekoding blir enklere. Raskere linking ved hopp, færre bits trenger i instruksjoner fordi adressene er nærmere hverandre. Betingede hopp (beq, blt, «Hopp hvis, dersom dette...») Ubetingede hopp (j, jal, jalr, «Bare hopp»)		
--	---	--	--

T2.2: Heltall og logiske operasjoner	• Studenten skal kunne representere heltall som binære tall (grunntall 2) og heksadesimale tall (grunntall 16) samt kunne oversette mellom disse og tall i det desimale systemet (grunntall 10). Bare tenk • Studenten skal kunne representere negative heltall på 2's komplement form samt kunne oversette mellom dette formatet og tall i det desimale systemet. Mest signifikante bit er fortegn. For 8 bit: -127 ? Oversetting til tall i desimalsystemet bør gå bra. For å oversette til positivt tall: flipp alle bits, +1	• 2.10 • 2.11	<i>Ingen spesielt relevante</i>
---	---	--	---------------------------------

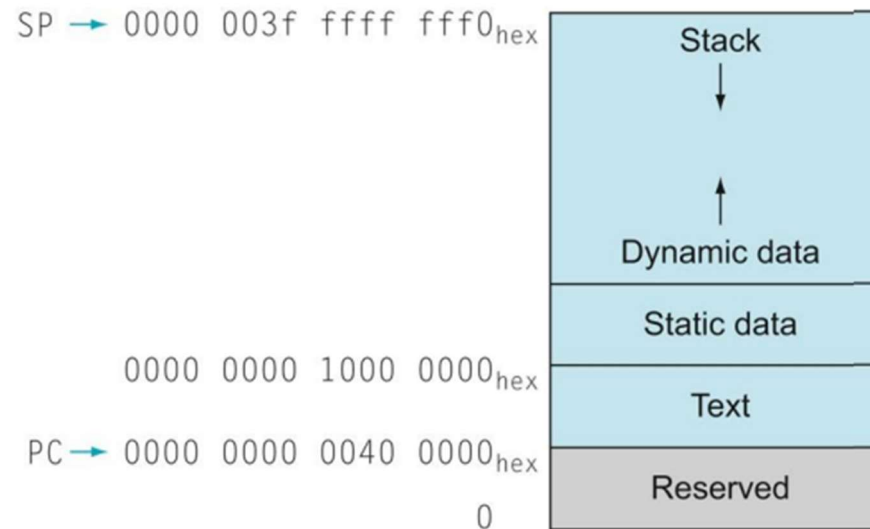
² Oppgavetyperen er relevant, men den ville vært vanskeligere nå enn da den ble gitt siden vi bruker RISC-V ISA i øvingene. Vedlegget gir imidlertid nok informasjon til at oppgaven skal kunne besvares.

	• Studenten skal forstå hvorfor fortegnsutvidelse av et tall på 2's komplement form beholder samme tallverdi. Fortegnssutvidelse: når vi har et tall med f.eks 8 bit og vil representere det med 16 eller 32 bit. For å beholde samme tallverdi så må man; + : Fylle inn med 0 - : Fylle med 1 • Studenten vite hva overflyt er og forstå når det oppstår. Når vi prøver å representere et større tall enn vi har bits til. Dette kan skje dersom man f.eks. prøver å plusse på et tall som er veldig stort. Eks. 1111 1111 + 1 = (1) 0000 0000 der (1) er den 9ende biten som vi ikke har maskinvare til. Opp til programmet, operativsystemet og programmeringsspråket å bestemme hvordan dette håndteres. Legge sammen 2 positive tall og få et negativt tall, eller Legge sammen 2 negative tall og få et positivt tall. Aldri når vi legger sammen et positiv og et negativt tall • Studenten skal kunne utføre logiske operasjoner på binære tall, inkludert bitvis AND og OR samt logisk og aritmetisk bitskifting. Det er vel bare å sette de over hverandre og tenke litt. Bitskifting er å flytte bitene til høyre eller venstre og fylle: Logisk skift: bevarer ikke fortegn, fyller med 0 uansett Aritmetisk skift: bevarer fortegn, fyll med 0 (+) eller 1 (-)		
T2.3: Funksjonskall	• Studenten vet hvilke oppgaver som skal utføres ved et funksjonskall og kan forklare konseptet kallkonvensjon. Ved funksjonskall: 1. Legg parameterne et sted funksjonen kan få tak i dem 2. Overfør kontroll til funksjonen (j, jal, beq...) 3. Allokere nødvendige ressurser 4. Utfør oppgaven 5. Legg returverdien et sted der den som kalte funksjonen kan få tak i dem 6. Overfør kontroll til kallstedet Må huske hvor funksjonen ble kalt fra fordi funksjoner typisk kalles fra mange ulike steder i koden. (ra?)	• 2.30 • 2.31 • 2.32	<i>Ingen spesielt relevante</i>

Caller: programmet som kaller en funksjon. Utfører steg 1-3

Callee: funksjonen som kalles. Utfører 4-6.

- Studenten kjenner RISC-V minnekartet (kan gjengi og forklare figur 2.13).



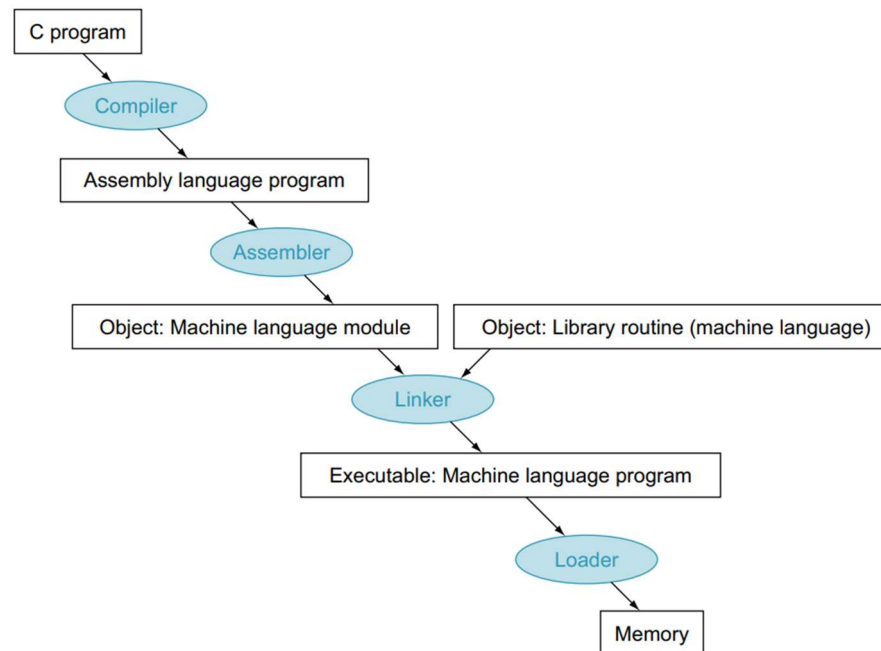
Figur 2.13

Dette er RISC-V-konvensjonen for tildeling av minnet, ikke en faktisk innebygd del av arkitekturen.

- **Reservert område:** reservert for operativsystemet, unntak, spesifikke systemfunksjoner osv.
- **Tekstsegmentet (code):** Inneholder RISC-V maskinkoden for rutiner i kildekoden. Instruksjonene til programmet: funksjonskall, hopp, branch, osv.
- **Static data:** Konstanter og andre statiske variabler. Statiske data er tildelt for hele programmets levetid. Kan endre variablenes verdi, men de har faste adresser gjennom programmet.
- **Dynamiske data (heap):** Brukes for datastrukturer som vokser og krymper ilt levetiden, som linked list. Vokser oppover mots stacken. Har ingen fast

	størrelse. - Stack: Der man lagrer midlertidige registerdata. Vokser nedover i minnet. Stack pointer hjelper med å fortelle hvor neste data skal lagres, og justeres med ett ord, -4, for hver nye lagrede verdi. Last in first out. Push. Når vi fjerner et element fra stacken, pop, justeres sp med +4. Lagring av bevarte registre, temporære verdier og evt. Funksjonsargumenter. Hvis stacken og heapen møtes, får vi overflow. fp, frame pointer kan brukes for å peke på første ordet på stacken i den nåværende funksjonens minneområde, sp peker på siste ordet i nåværende funksjons minneområde. Registere som må lagres: Alle sN, sp, fp, ra. • Studenten forstår forskjellen på statiske og dynamiske data. - Dynamisk data: bestemmes mens programmet kjører. Eks. nye data som allokeres til - Statiske data: bestemt når programmet kompiles. De eksisterer gjennom hele programmets levetid. Eks. globale variabler. Ved funksjonskall: Steg 1, 3, 5 handler om å allokere og deallokere statiske data		
T2.4: Instruksjoner, diverse	• Studenten kjenner til hvordan tekst representeres i en datamaskin. Bokstaver representeres som tall (char fra c++). ASCII: 8bit, 1byte per bokstav Unicode: 2 byte Kan hentes med lbu, load byte unsigned (1byte), eller lhu, load halfword unsigned (2 byte) • Studenten skal vite hvordan vi håndterer store konstanter og (unngår) lange hopp (PC-relativ adressering). Store konstanter: U-type: 20 bit immediate. Hvis dette er for lite: Lui, load upper immediate, setter bitene [31:12] Addi kan plusse på de 12 nederste bit-ene på de øverste bitene. Eks:	• 2.22 • 2.34	Ingen spesielt relevante

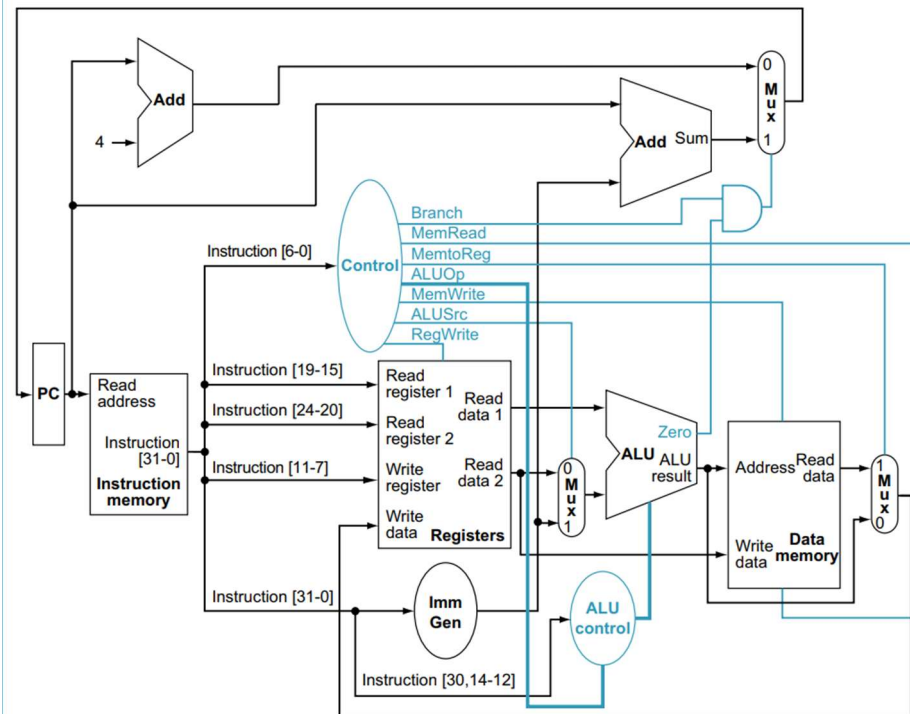
	Lui t0, 0xAAf32 Øverste 20 bit Addi t0, t0, 0x345 Nedre 12 bit. Lange hopp: Tilsvarende, auipc, add upper immediate to PC, øverste 20 bit. Baseadresse relativ til PC. auipc t0, 0xAAF12 # legg til store offset på PC (20 bit) jalr s0, 0x345(t0) # Legger til nedre 12 bit som offset • Studenten skal kunne forklare RISC-Vs fire adressemodi. - Konstantbasert adressering: Eks. addi a0, t0, 8. a0 er da adressen til t0, flyttet med 8 bytes. - Registerbasert adressering: Verdien som brukes som operand ligger i et register. Eks. add t0, t2, a2 - Grunnadressebasert adressering: Base + offset adressering. Lw t0, 5(a0), base er a0, 5 er offset. - PC-relativ adressering: PC + offset. Brukes i hopp, forgreininger, lange hopp (auipc + jalr) Hvis vi har flere prosessorer og vil sikre at bare én gjør noe av gangen, kalles det synkronisering. Koden som må utføres alene er en kritisk seksjon. Kan bruke load reserved lr.w og store contidional for å passe på at dette går smuud. • Studenten skal kunne forklare skrittene involvert i oversettelse og oppstart av programmer (kunne gjengi og forklare figur 2.20)		
--	--	--	--



- Programmet **kompileres** til assembly. **Krysskompilering**: kompilerer for et annet instruksjonssett enn maskinen du er på. Kompilerer på en maskin for en annen arkitektur. Eks. fra x86 til x32.
- **Assembler** oversetter til 1 og 0. **Pseudoinstruksjoner**: instruksjoner assembleren forstår, men som ikke eksisterer i instruksjonssettet. Byttes ut av ekte instruksjoner i assembleren.
Eks: j lokke -> jal x0, offset. Mv t1, t2 -> addi t1, t2, 0
- **Linker** kombinerer vårt program med libraries og annen kode «utenfor» det vi har skrevet og definert (i 0 og 1), samt kode fra andre kodefiler. Fikser opp i referanser og sånt, slår alt sammen til ett program (en **executable**). Dette gjøres da på en logisk måte, og så legger den inn riktige adresser for funksjoner og variabler.

Statisk linking: Linkeren kopierer delte bibliotek inn i en .exe. Problem: Da kan ikke bibliotekene oppdateres uavhengig av programmet og biblioteksfunksjonene

	lastes inn uavhengig om de blir brukt eller ikke. Dynamisk linking: Tabell i minnet med pekere til biblioteksfunksjonene programmet kan bruke. Første aksess: en funksjon laster inn bib.funksjonene, oppdaterer pekeren til å peke til riktige bib.funksjoner og kaller bib.funksjonene. Alle senere kall går direkte fra tabellen til bib.funksjobnen. - Loader leser .exe-filen, <u>allokerer</u> riktig mengde plass i minnet for all informasjonen og <u>kopierer</u> instruksjonene og dataen fra exe-fila til minnet. Kopierer eventuelle parametere til stacken. <u>Initialiserer</u> registerne og flytter sp. <u>Brancher</u> til oppstartsrutinen og kaller på «main»-rutinen til programmet. Når programmet returnerer, <u>stopper</u> oppstartsrutinen programmet med «exit»-kall.		
T3.1: Enkeltskykel-prosessor	• Studenten skal kunne forklare begrepene datasti og kontrollenhet. - Datasti er veien som instruksjonene og dataene går. Denne skal gå gjennom forskjellige moduler slik at alle instruksjoner kan løses. - Kontrollenheten bestemmer kontrollsignalene basert på operasjonskodene i instruksjonen. Kontrollsignalene bestemmer hvilke enheter som skal være aktive. Har ofte 2 kontrollenheter: Control og ALU control. Det er fordi disse svarer til gigantiske sannhetstabeller som hadde vært enda mye større hvis de var slått sammen. Sparer plass, forenkler, kortere kritisk sti? • Studenten skal kunne forklare mikroarkitekturen til enkeltskykelprosessen (kunne gjengi og forklare figur 4.21).	• 4.1 • 4.3 • 4.5 • 4.7	• Kont 25: Oppgave 2 • Ord. 24: Oppgave 2 • Ord. 23: Oppgave 5 • Kont 23: Oppgave 3



- Kikk på denne

- Studenten skal kunne sette opp rett kontrollord for en instruksjon, inkludert korrekt bruk av «don't care».
- Dont care: mem to reg for sw og beq. IKKE BRUK dersom det kan føre til uønsket tilstandsendring i registerfila eller hurtigbuffer. Aldri i memRead, RegWrite, memWrite.
R-type og I-type: memToReg x

Øv litt på dette?

- Studenten skal kunne identifisere instruksjonstype fra et kontrollord.

Øv

T3.2: Kombinatorisk logikk	• Studenten skal kjenne til de logiske portene AND, OR, og inverter og deres symboler samt kunne beskrive oppførselen deres med sannhetstabeller. ADE • Studenten skal kjenne til symbolene og kunne gjengi sannhetstabellene for kombinerte logiske porter (NAND og NOR) og mer avanserte kretser (multiplekser og dekode). ADE • Studenten skal kunne oversette mellom boolske uttrykk på sum-av-produkt form og sannhetstabeller. Sum-av-produkt er bare logisk uttrykk. ADE • Studenten skal kjenne til konseptet buss og notasjonen for adressering av enkeltlinjer i busser. Buss: felles fysisk eller logisk kommunikasjonsmedium mellom flere enheter. Typ ledninger. Adressering av enkeltlinjer: ----instruksjon[31-23] for eksempel. Flere bits i en ledning/buss.	• A.5 • A.7 • A.8 • A.10 • A.11	• Kont 24: Oppgave 2 • Ord. 23: Oppgave 3 • Kont 23: Oppgave 2 • Ord. 22: Oppgave 4 og 5 • Kont 22: Oppgave 3
-----------------------------------	--	---	---

T3.3: Aritmetisk-logisk enhet

- Studenten skal kunne konstruere en 32-bit aritmetisk-logisk enhet (ALU) som kan gjøre addisjon og subtraksjon samt logisk AND og OR og nulldeteksjon (gjengi og forklare figurene A.5.7 og A.5.8).

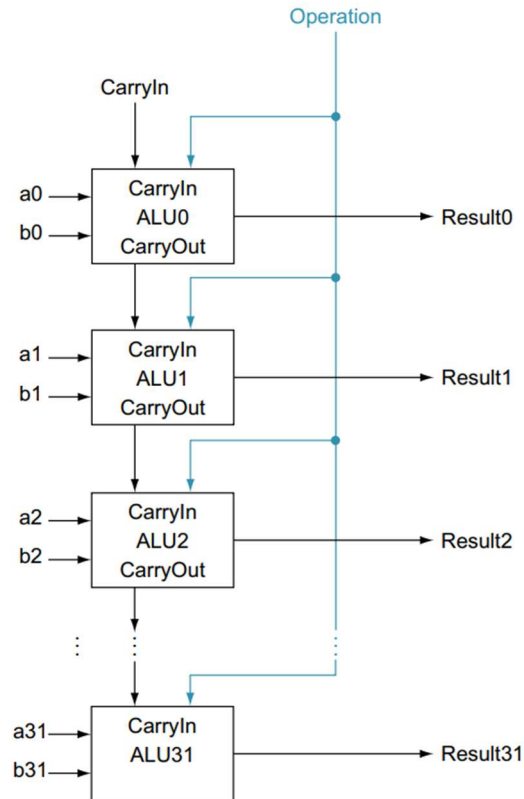


FIGURE A.5.7 A 32-bit ALU constructed from 32 1-bit ALUs. CarryOut of the less significant bit is connected to the CarryIn of the more significant bit. This organization is called ripple carry.

32 bit ALU. Tar inn ax, bx, velger operasjon og får inn en carry in (fra operasjonen for den mindre signifikante biten eller utenfra hvis den er først). Gir ut et resultat. Hver boks kan lages på denne måten:

- 3.3
- 3.6 til 3.8
- 3.12
- 3.18
- 3.20 til 3.23

- Ord. 22: Oppgave 6 og 7

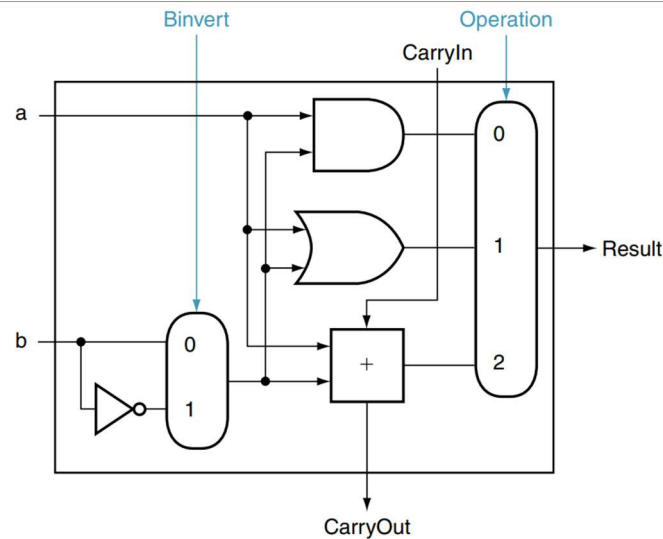


FIGURE A.5.8 A 1-bit ALU that performs AND, OR, and addition on a and b or a and $\bar{b}$. By selecting $\bar{b}$ ($\text{Binvert} = 1$) and setting CarryIn to 1 in the least significant bit of the ALU, we get two's complement subtraction of b from a instead of addition of b to a .

Hvis vi har $\text{Binvert}=1$ og $\text{carryIn}=1$ så kan vi skrive B som negativt på 2-komplement form, slik at vi kan bruke minus. Operation-bussen i forrige tegning vil da være 2 bits, Binvert og Operation .

Beq: trenger å vite om $A-B=0$. Kan da legge til et stor OR-port med alle resultatene og en inverter, slik at hvis $A-B=0$, blir «zero»-utgangen 1.

Blt/Bgt: Sjekke om sign-bit er 0 eller 1 etter å ha gjort $A-B$.

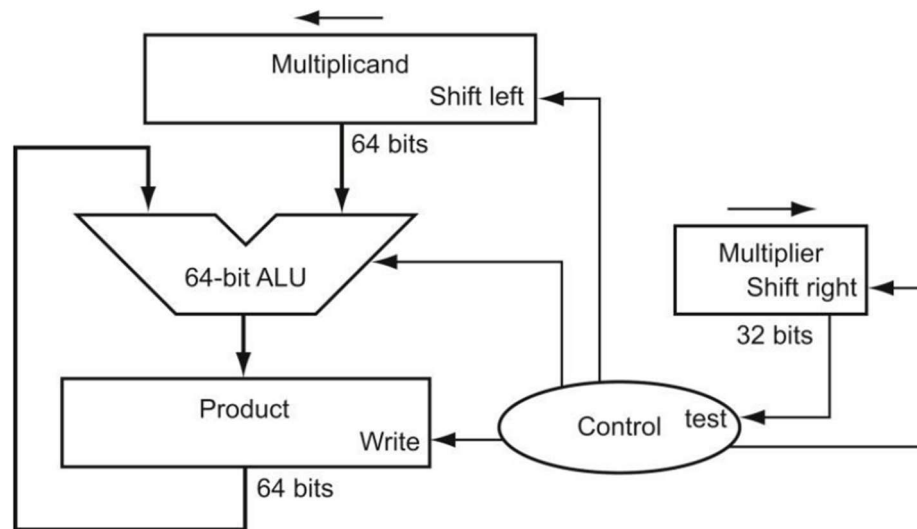
Detektere overflyt: Kan ta xor av carry in og carry out fra mest signifikant bit (ikke sign-bit). Hvis jeg har c_{in} og c_{out} , så har jeg overflow. Hvis ikke får jeg ikke overflow.

Denne ALUen er dårlig og treg. «Carry look-ahead adder»: mer maskinvare, kan regne menteverdiene parallellt. Litt mer komplisert. Noen ting kan bestemmes fortere enn å ta ting fra bunnen av.

- Studenten skal kunne utføre multiplikasjon og divisjon av binære tall og

beskrive hvordan disse implementeres i maskinvare.

Multiplikasjon



Tenk multiplikasjon for hånd:

- Sjekk nederste bit av multiplier:
- Hvis biten =1 -> legg til multiplicand i produktet
Hvis biten = 0 -> gjør ingenting
- Skift multiplicand til venstre (gange med 2)
 - Skift multiplier til høyre (dele på 2)
 - Gjenta

Skifter multiplicand til venstre fordi vi går gjennom bitene fra LSB -> MSB.

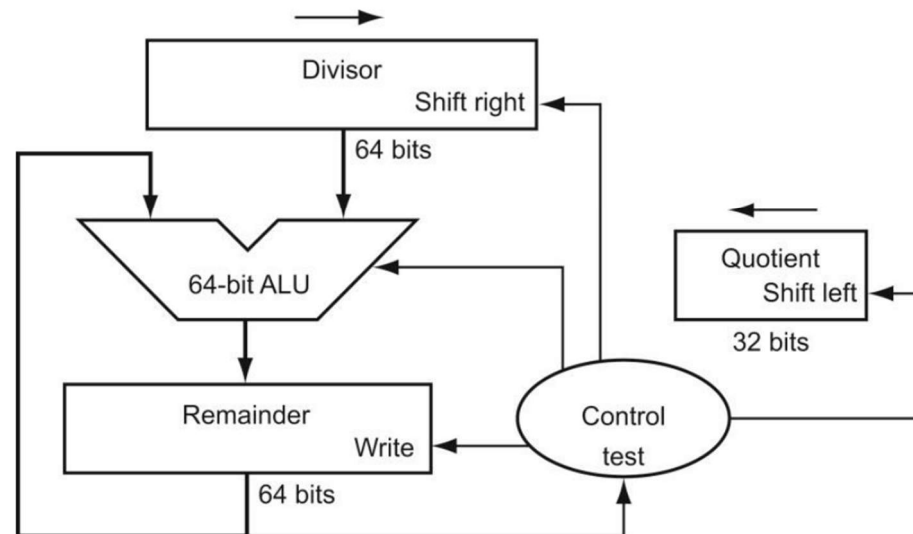
Skifter multiplier til høyre slik at neste bit kommer inn i LSB.

Annen måte å se det på: Tar ett og ett tall i multiplier og ganger det med multiplicand (gange med 1 eller 0, AND). Skifter multiplicand fordi ved neste bit ganger man med noe som er større, typ.

Høyere ytelse: Et tre av adderere, kan ta ca. 3 klokkesykler i gode prosessorer

Divisjon:

$$\text{Dividend} = \text{Kvotient} \cdot \text{Divisor} + \text{Rest}$$



Mye likt, men har løkke med subtraksjon, men skal bare trekke fra dersom rest-verdien er større enn 0. Må også håndtere divisjon med 0 ikke lov.

Dividend / divisor = kvotient

Dividend går inn i remainder fordi det er det som er det foreløpige resten.

- Start: Dividend går inn i reminder-registeret.
- Divisor-registeret lastes med divisoren, men plassert slik at MSB som er 1, matcher dividendens MSB.
- Quotient = 0

For hver bit:

- ALU tar remainder - divisor
- Sjekker sign-bit (fortegn):
Hvis sign = 0: skriv 1 som neste kvotient-bit, og oppdater remainder
Hvis sign = 1: 0 som neste kvotient-bit, og ikke skriv den nye verdien i Remainder-registeret.
- Control tar kvotient-biten fra forrige steg, skifter quotient-registeret 1 til

- venstre og legger inn den nye biten som LSB
- Divisor-registeret skiftes høyre, tilsvarende som man gjør når man regner for hånd.
 - Gjenta

Rest på slutten må håndteres, samt divisjon på 0 ikke er lov.

Høyere ytelse: Et tre av adderere, men hvert skritt er avhengig av den nye verdien i «Remainder» registeret. Tar noen titalls klokkesyklus.

KIKK PÅ DISSEEE ????? ØV

- Studenten skal kunne oppgi fordeler og ulemper med å representere tall i et «fixed-point» format.

Fixed point: Representere desimaltall som 2^{-n} . Så 1011 kan bli $2^1 + 2^{-1} + 2^{-2}$.

Fordel: Kan regne som vanlig heltall, int. Trenger ikke mer maskinvare. Må bare omdefinere verdien til hver bit.

Ulempe: Må velge mellom å representere store tall og små tall.

Løsning: float!

- Studenten skal kunne motivere for hvorfor vi trenger flyttall og beskrive prinsippene for hvordan addisjon og multiplikasjon med flyttallsverdier utføres i maskinvare.

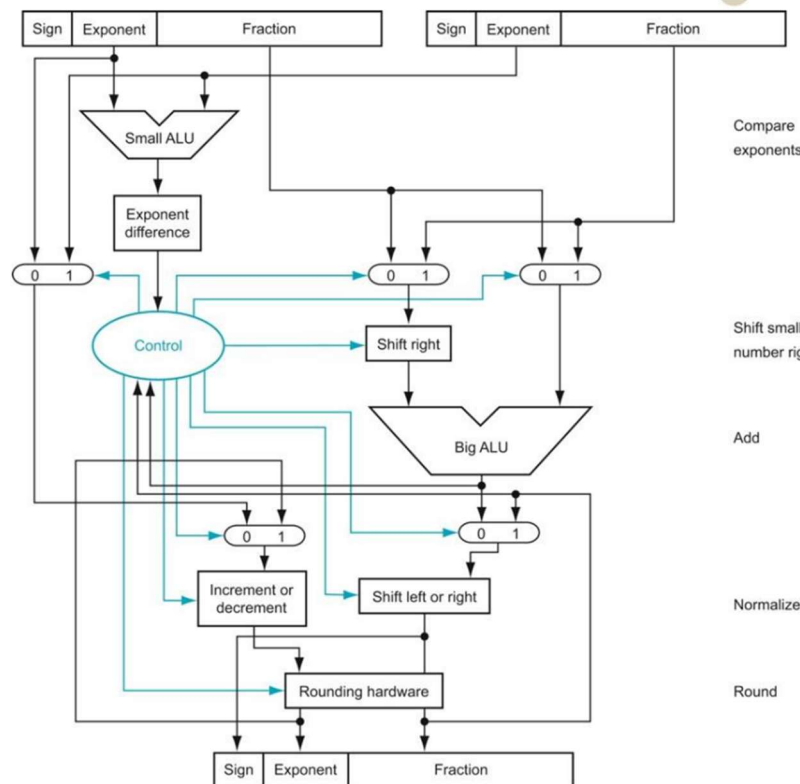
Flyttall - basert på vitenskapelig notasjon. Tallformat der komma flytter seg ved behov.

1 bit fortegn (s), 8 bits eksponent, 23 bits fraction
 $(-1)^s * fraction * 2^{exponent}$

Vanlig format:

$(-1)^s * (1 + fraction) * 2^{(exponent - bias)}$ der bias er 127 / 1023 for enkel / dobbel presisjon.

	Kan representere veldig store og veldig små tall. Krever mer maskinvare for å regne på. Overflow og underflow: når tallet er for stort / lite i + eller - retning til å representeres med exponent. Maskinvaren for addisjon er litt mer komplisert. Her er essensen: - Regn ut forskjell mellom exponentene (korrigjer for bias) - Skift den minste operanden til høyre for å få kommaene til begge tallene på samme sted - Legg sammen (1 + fraction) til begge tallene - Skift resultatet slik at fraction normaliseres, 1.xxx Multiplikasjon: - Legg sammen eksponentene og korrigjer for bias - Multipliser (1+fraction)-verdiene - Normaliser, 1.xxx		
--	---	--	--



Studenten skal kunne konvertere fra desimaltall til flyttall og motsatt.

Øv på egenhånd

Desimaltall til flyttall:

- Finn sign (fortegn, lett)
- Exponent:
Finn høyeste 2er-eksponent n som kan brukes for å representere tallet.
Exponent er da: $n = E - 127 \rightarrow E = n + 127$
- Fraction:
 $\text{Tallet} / 2^{(E-127)} = 1 + F$
Løs for F

F * 2. Hvis det er under 1: skriv 0
Større enn 1: skriv 1 og ta tallet minus 1
Repeter til F * 2 == 1

Flyttall til desimaltall: putt inn i formelen ovenfor

- Studenten skal kjenne til hvordan man implementerer SIMD- instruksjoner og hva de brukes til.

Single input, multiple data. 1 instruksjon utfører samme operasjon på flere dataelementer samtidig.

GPU. ØV, SJEKK UT, FINN UT

T4.1: Flersykel- prosessor

- Studenten skal kunne forklare mikroarkitekturen til en flersykelprosessor (kunne gjengi og forklare figur e.4.5.4).

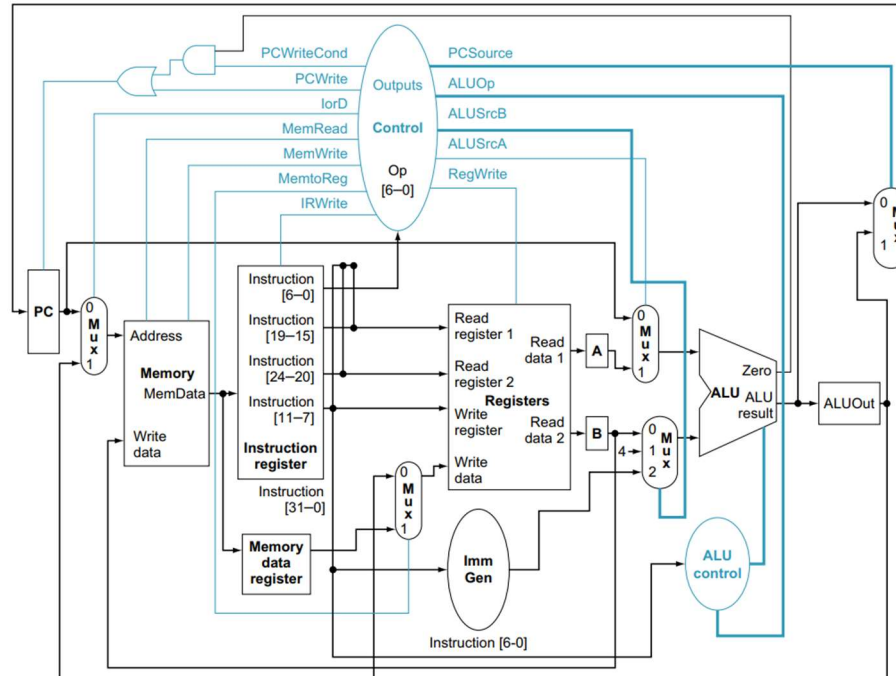


FIGURE e4.5.4 The complete datapath for the multicycle implementation together with the necessary control lines. The control lines of Figure e4.5.3 are attached to the control unit, and the control and datapath elements needed to effect changes to the PC are included. The major additions from Figure 4.29 include the multiplexor used to select the source of a new PC value; gates used to combine the PC write signals; and the control signals PCSource, PCWrite, and PCWriteCond. The PCWriteCond signal is used to decide whether a conditional branch should be taken.

Deler instruksjonene i flere steg:

- **IF, instruction fetch.** Instruksjonen hentes fra memory i adressen til PC til Instruction register. Det sendes også til MDR. PC oppdateres
- **ID, instruction decode.** Instruksjonen dekodes, Dataen går til Registers. Immediate Gen lager en immediate-verdi. Genererer kontrollsignaler for videre steg.
- **EX, execute.** ALU beregner, adresseregning med offset for minneoperasjoner, branch-evaluering og testing av branch-betingelser. Verdien blir lagret i ALUout
- **MEM, memory access.** Load: leser fra minnet på adressen, Store skriver registerverdi til data-minnet i Memory, fra ALUout. Merk at det bare er

Ingen
tilgjengelig
e oppgaver

- Ord. 24:
Oppgave 3

minneinstruksjoner som går gjennom dette steget.

- **WB, write back:** Skriver resultatet tilbake til registerne.

Trenger noen ekstra registre for å lagre verdier mellom klokkesykler:

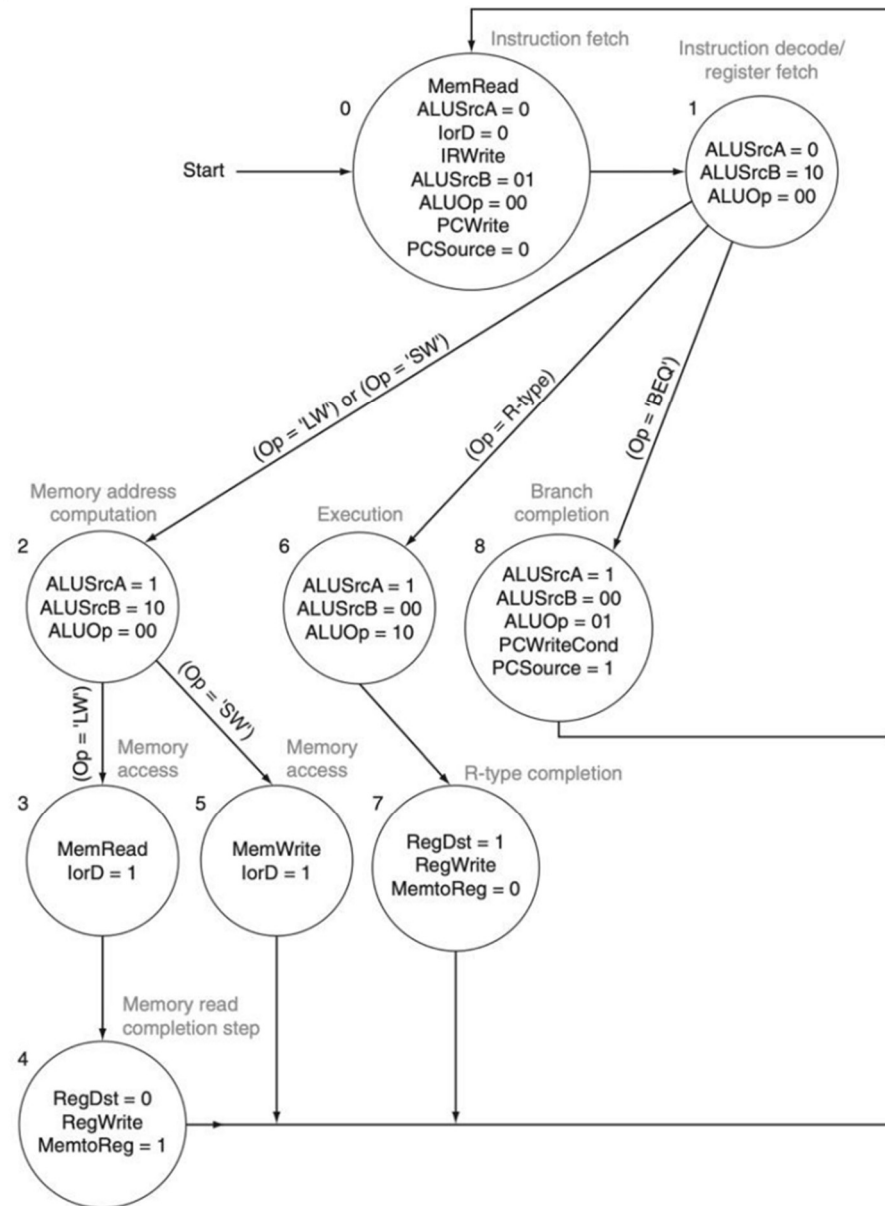
- **Instruction register:** Lagrer instruksjonen i IF-steget slik at samme instruksjon kan brukes i flere sykler. Første syklus går til å få instruksjonen inn, neste syklus brukes til å sende videre til kontrollenheten, «Registers» osv.
- **Memory data register:** Lagrer data som kommer fra minnet, eller data som skrives ut til minnet, fordi minnet ikke leverer data momentant. MemToReg. Eks: lw a0, 8(a5), verdien i a0 går inn i MDR, som etter en sykel skrives inn i register a5
- **A:** Lagrer første operand lest fra registerfila i decode-steget.
- **B:** Lagrer andre operand. Brukes som
- **ALU out:** Lagrer resultatet fra ALU slik at det kan brukes senere.

Datasti og kontrollenhet:

- **lorD:** velger adressekilde for minnet: PC eller ALUOut
- **MemtoReg:** Bestemmer om vi skal hente minneadressen i MDR eller fra ALUout

ØV! Spesielt på kontrollsignaler

- Studenten skal kunne forklare hvorfor kontrollenheten til flersykelprosessen blir en tilstandsmaskin.
- Én instruksjon utføres over flere klokkesykler, der hver syklus gjør en forskjellig del av arbeidet
- Instruksjonene har forskjellig lengde
- Hver syklus krever nye kontrollsignaler
- Prosessoren må vite hva som ble gjort i forrige syklus for å vite hva den skal gjøre nå. Dette krever hukommelse



- Studenten skal kunne beskrive sammenhengen mellom kontrollordet til flersykelprosessoren og tilstanden i tilstandsmaskinen.

	Sammenheng med å øve på kontrollord. • Studenten skal kunne beskrive fordeler og ulemper ved flersykelprosessen sammenliknet med enkeltsykelprosessen. Bruker flere klokkesykler på en instruksjon, CPI øker. Mindre arbeid per klokkesykel -> klokkefrekvensen øker. De kan bli veldig små og billige Ganske dårlig ytelse. Problemer med enkeltsykelprosessen: Lang kritisk sti, må ha flere adderere og minner fordi vi må gjøre alt i en klokkesykel. Vanskelig med divisjon, som ofte tar mange klokkesykler. Tar mer areal, uten at ytelsen går så mye opp. Alle instruksjoner tar like lang tid, som ikke gjør «common case fast»		
T4.2: Sekvensiell logikk	• Studenten skal kunne beskrive oppførselen til komponentene SR-lås, D-lås, og D-vippe. SR-lås: Asynkron, tar ikke inn clk. 2 innganger, set og reset. Har utgangene Q og Qikke. S=1, R=0 gir Q=1 S=0, R=1 gir Q=0, S=0, R=0 gir Q= forrige verdi/tilstand Ugyldig med S=R=1 2 nand-gates med utgang som går inn i hverandre, typ D-lås: Synkron, tar inn D og clk. Clk=1 - Q følger D Clk=0 - Q holder siste verdi. Som D-vippe, men oppdateres mens klokkesignalet er høyt i stedet for på stigende klokkeflanke. Består av en SR-lås med klokkesignal inni 2 And-gates der D og D-ikke går inn D-vippe: Oppdateres på stigende klokkeflanke. Består av 2 etterfølgende D-låser	• A.36 • A.37 • A.38 • A.39	• Ord. 23: Oppgave 2 • Ord. 22: Oppgave 2 og 3 • Kont 22: Oppgave 2

med clk og clk-ikke i clk-portene

- Studenten skal kunne konstruere registre og registerfiler med kombinatoriske og sekvensielle logiske komponenter (kunne gjengi og forklare figur A.8.8 og figur A.8.9).

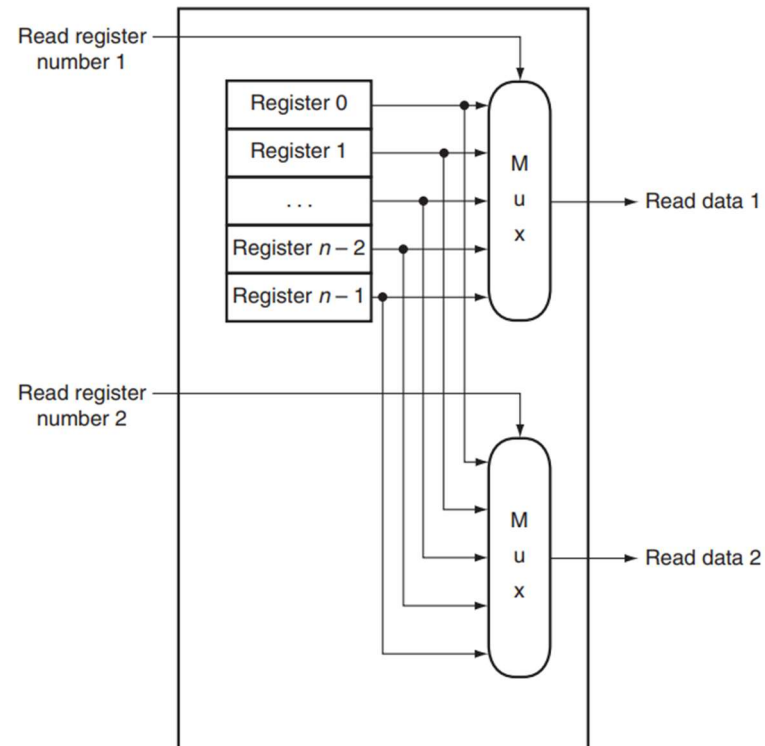


FIGURE A.8.8 The implementation of two read ports for a register file with n registers can be done with a pair of n -to-1 multiplexors, each 32 bits wide. The register read number signal is used as the multiplexor selector signal. Figure A.8.9 shows how the write port is implemented.

Hvert register består av D-vipper. Har masse registre, bruker MUX for å hente ut registerdata på register nr. i . Denne er ganske grei.

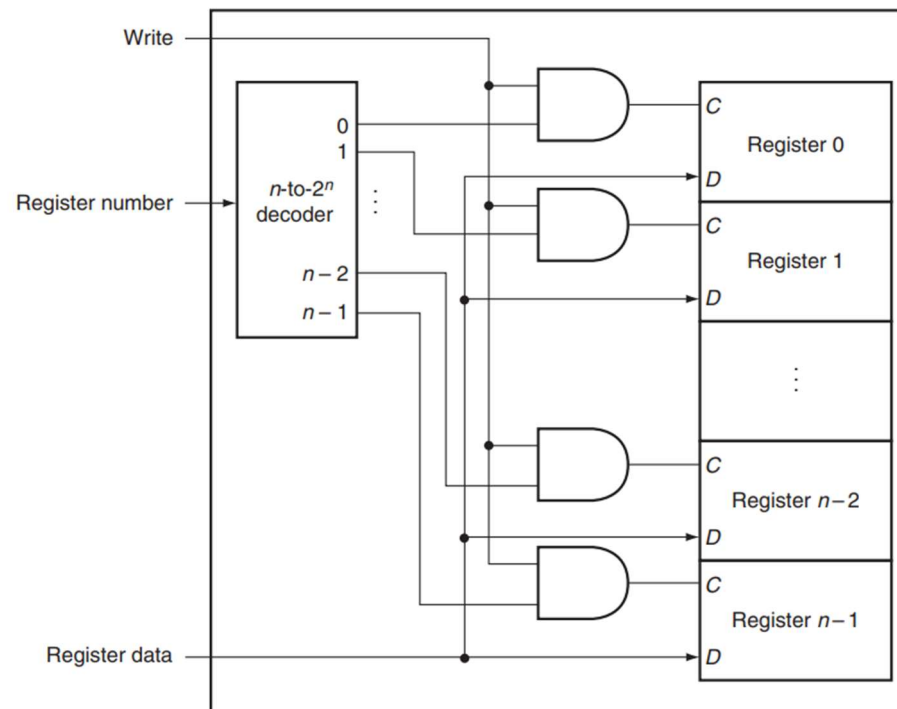


FIGURE A.8.9 The write port for a register file is implemented with a decoder that is used with the write signal to generate the C input to the registers. All three inputs (the register number, the data, and the write signal) will have setup and hold-time constraints that ensure that the correct data are written into the register file.

Legger register-data inni alle D-inputs til registerne, som ikke oppdateres dersom de ikke får en stigende flanke på C.

Har en dekode som endrer register-nummeret i bits til én av utgangene. Eks. hvis reg.num er 0101, så vil kun utgang 5 bli høy. Den går da inn i en AND-gate med kontrollsignalet for write, som vil gi høy kun for ett register. Da får den en stigende flanke, som gjør at register-dataen skrives inn.

- Studenten skal kunne forklare hva en tilstandsmaskin er og beskrive hvordan den implementeres i maskinvare (kunne gjengi og forklare figur A.10.3).

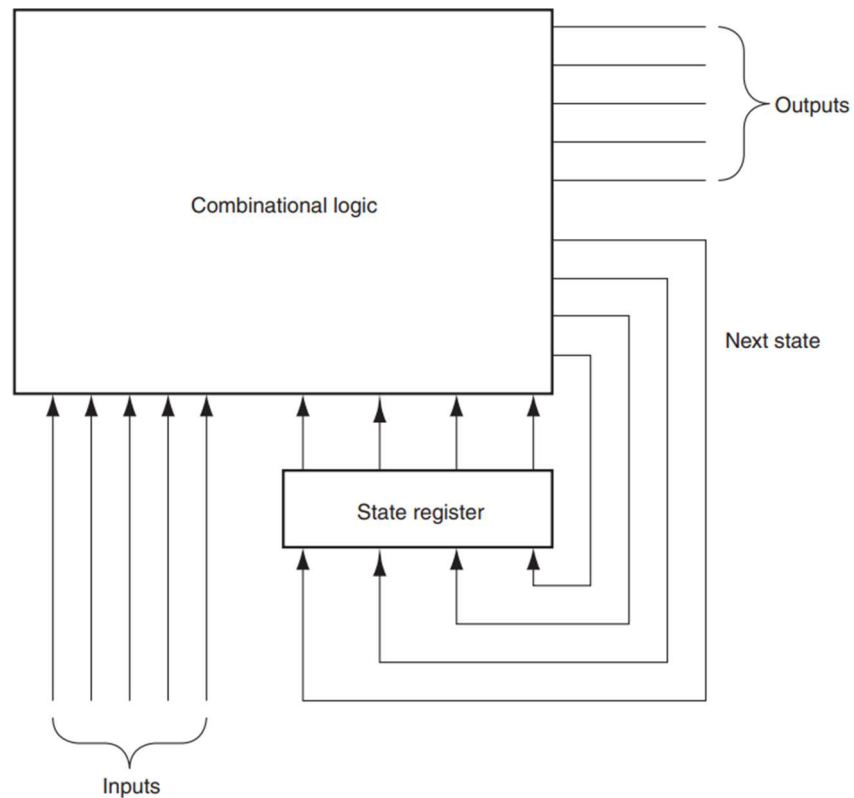


FIGURE A.10.3 A finite-state machine is implemented with a state register that holds the current state and a combinational logic block to compute the next state and output functions. The latter two functions are often split apart and implemented with two separate blocks of logic, which may require fewer gates.

Utgangene er avhengig av inputs og utgangene til state register. Avhengig av inputs og hvilken state vi er i, blir neste state bestemt, som påvirker neste output.

Kan bruke kominatorisk logikk, registre for state register og lage et tilstands-diagram. ESDA

- Studenten skal kunne forklare funksjonen til klokkesignalet i en datamaskin og begrepet «kritisk sti».

	Registere lages av D-vipper, som krever at signalet er stabilt rundt klokkeflanken. Klokkesignalet passer på at D-vippene, og registerne, oppdateres når signalet har kommet gjennom kombinatorisk logikk og stabilisert seg, slik at systemet oppdateres synkront og ikke uønskede transienter forårsaker feil og endringer. Kritisk sti: Den datastien som tar lengst tid.		
T5.1: Samlebånds-prosessor med 5 steg	• Studenten skal kunne beskrive fordeler og ulemper ved samlebåndsprosessen sammenliknet med flersykelprosessen og enkeltsykelprosessen. Enkeltsykelprosessen: Lang kritisk sti, høy klokkefrekvens, lav ytelse. Må ha flere adderere og minner fordi alt gjøres i samme klokkesykel. Alle instruksjoner tar like mye tid. Flersykelprosessor: Kan øke klokkefrekvensen pga mindre arbeid per sykel. Forenkler datasti fordi vi kan gjenbruke enheter mellom klokkesyklar. CPI øker. De kan bli veldig små og billige. Ganske dårlig ytelse Samlebåndsprosessor: Utnytter parallellitet, jobber med flere instruksjoner samtidig. Kan ta mye areal og bli veldig kompleks, som øker prisen. Lav CPI og høy klokkefrekvens gir god ytelse. • Studenten skal kunne forklare hvordan oppstartskostnad og balanse mellom steg i samlebåndet påvirker ytelse. Oppstartskostnad, må fylle samlebåndet før den blir ordentlig effektiv. Flere steg i samlebåndet gir høyere oppstartskostnad, men har også andre fordeler. Alle instruksjoner må gjennom alle steg, og alle steg må ta ca. like lang tid. Ellers må klokkefrekvensen senkes og ytelsen går ned	• 4.16 • 4.17 • 4.18 • 4.22-4.25	• Kont 25: Oppgave 3 • Ord. 24: Oppgave 4 • Kont 24: Oppgave 3

	Balanse: Kompromiss mellom lengde på samlebånd (flere steg, mindre jobb per sykel -> høyere clk) og oppstartskostnad. Balanse på alle steg i samlebåndet: alle steg må ta like lang tid.		
--	---	--	--

	• Studenten skal kunne beskrive forskjellen mellom avhengigheter og farer samt gjengi de tre hovedgruppene av farer (strukturfarer, datafarer, og kontrollfarer). Avhengigheter: Når en instruksjon er avhengig av resultatet eller data fra en tidligere instruksjon. RAW, WAW, WAR Farer: Oppstår når et program ikke kan utføres fordi det «mangler noe». Oppstår når avhengigheter ikke kan håndteres korrekt eller i tide av pipelinen. Avhengighet er en egenskap ved programmet, mens en fare er en avhengighet som påvirker utføringen av programmet på en gitt maskin. • Strukturfare: En enhet som instruksjonen trenger er ikke tilgjengelig. Kan oppstå når to instruksjoner konkurrerer om samme maskinvare (f.eks. lese og skrive minnet samtidig). • Datafare: Instruksjonen kan ikke gjennomføres fordi data ikke er tilgjengelig. Oppstår når avhengigheter ikke håndteres direkte (RAW, WAW, WAR) • Kontrollfare: Vi vet ikke hvilken instruksjon som skal utføres. Oppstår ved hopp/branch-instruksjoner • Studenten skal kjenne til de fire overordnede strategiene for å håndtere farer (unngåelse, videresending, stans, og prediksjon) • Unngå ressurskonflikter: Gjøre slik at farer ikke oppstår. F.eks. Legge til programvare som gjør at vi kan lese to registre og lese ett register hver klokkesykel. Typisk mot struktur- og datafarer. • Videresending, forwarding: Legge til maskinvare som sender data direkte til instruksjonen som trenger dem. Eks. Legge til registre ved hvert steg som kan sende data fra ett steg til et annet. Må også lagre registernumrene i samlebåndsregisterne og legge til logikk som velger riktig verdi. MUX-er på ALU-inngangene slik at de kan ta imot verdier fra EX/MEM og MEM/WB Brukes mot datafarer. • Stans, stalling: Midlertidig stopp i pipeline frem til faren er løst. (bubbles/hull) Enkelt, men gir dårligere ytelse. Siste utvei.	• 4.27 • 4.28.1 og 4.28.2	• Ord. 23: Oppgave 4 • Kont 23: Oppgave 4 • Ord. 22: Oppgave 10 (litt relevant) • Kont 22: Oppg. 10
--	---	--	--

Brukes mot alle farer.

- **Prediksjon:** Vi gjetter: Antar at branches ikke tas, evt ha maskinvare som husker tidligere branch-resultater og gjetter basert på mønster. Holder pipelinen fylt og reduserer forsinkelser, men feil prediksjon krever «flush» av pipeline. Instruksjoner i pipelinen må kastes og man må tilbake til riktig instruksjon, som krever ekstra maskinvare. (tømming)
Brukes mot kontrollfarer.
- Studenten skal kunne forklare den grunnleggende mikroarkitekturen til 5-stegs samlebåndsprosessoren (kunne gjengi og forklare figur 4.43).

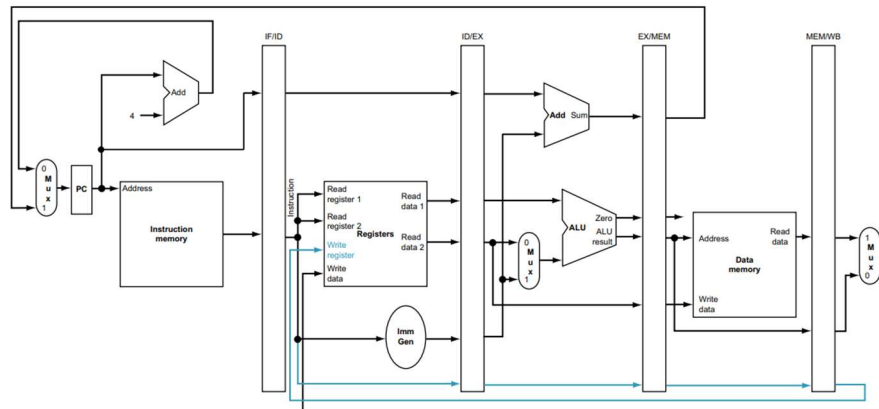


FIGURE 4.43 The corrected pipelined datapath to handle the load instruction properly. The write register number now comes from the MEM/WB pipeline register along with the data. The register number is passed from the ID pipe stage until it reaches the MEM/WB pipeline register, adding five more bits to the last three pipeline registers. This new path is shown in color.

Skriver til registre i 1. halvdel av klokkesyklus og Leser fra registre i 2. halvdel. Da unngår vi en fare. Da er den produserte verdien tilgjengelig før den må leses av en eventuell avhengig instruksjon.

- IF
- ID
- EX
- MEM
- WB

Samlebåndsregisterne lagrer dataverdiene instruksjonen trenger senere.

- Studenten skal kunne forklare hvordan kontroll implementeres i 5-steps samlebåndsprosessen.

I hvert samlebåndsregister legges det til registre som lagrer kontrollsignalene for denne instruksjonen. Etter hvert som vi har brukt kontrollsignalene og ikke trenger dem lenger, sendes de ikke videre. Slik sendes kontrollsignalene sammen med sine instruksjoner.

- Studenten skal kunne forklare hvordan videresending og stans kan brukes til å håndtere datafarer samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.

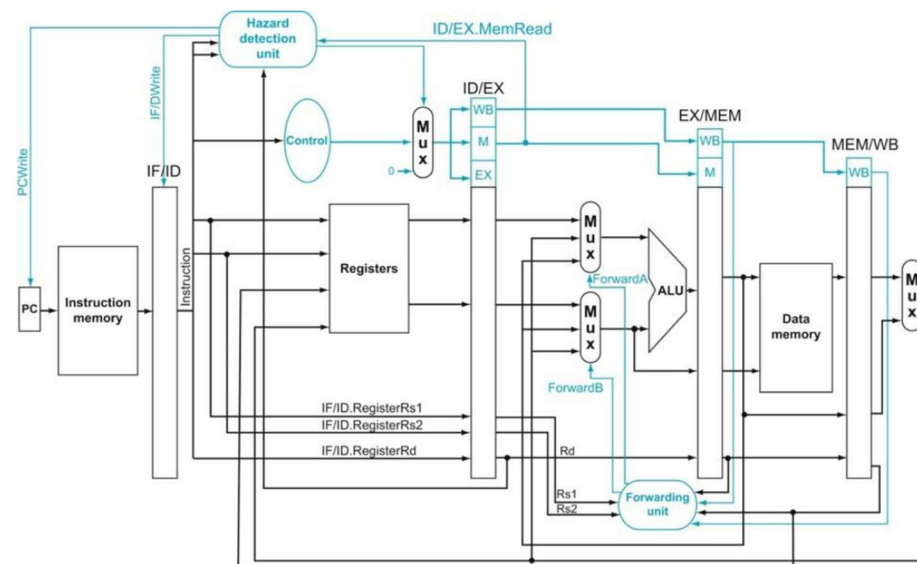
Videresending, forwarding: Legge til maskinvare som sender data direkte til instruksjonen som trenger dem.

Eks. Legge til registre ved hvert steg som kan sende data fra ett steg til et annet. Må også lagre registernumrene i samlebåndsregisterne og legge til logikk som velger riktig verdi.

MUX-er på ALU-inngangene slik at de kan ta imot verdier fra EX/MEM og MEM/WB

Brukes mot datafarer.

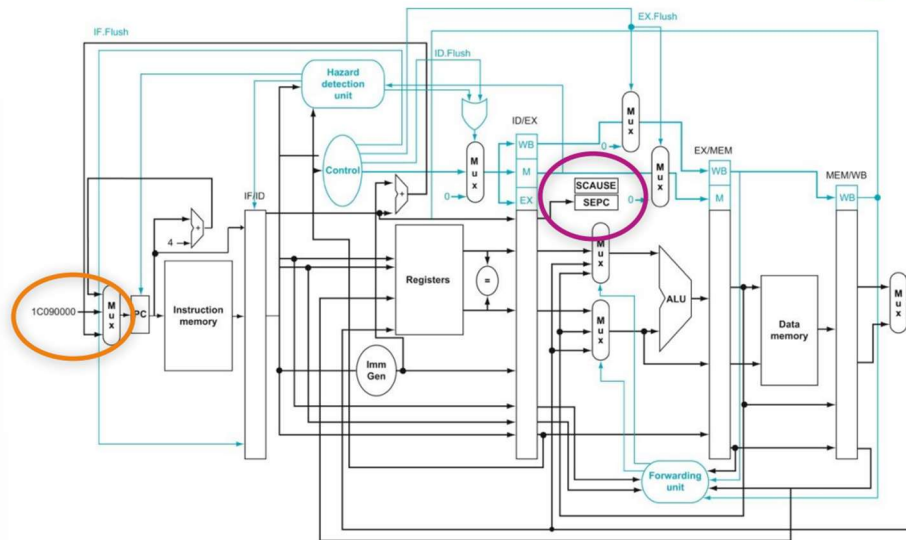
Eks:



Les-bruk farer: Videresending fungerer bare når verdien vi trenger har blitt produsert.

	Løsning: Stalling. Ha en hazard detection unit som oppdager dette. - Skal vi lese fra minnet? - Skal vi skrive til et register som neste instruksjon leser? - Ja på begge -> Lag hull, kontrollsignaler = 0, hold samme verdi i PC og IF/ID -> Mindre ytelse. • Studenten skal kunne forklare hvordan kontrollfarer kan håndteres med stans og prediksjon samt analysere hvordan strategiene påvirker prosessorens ytelse. Skjer ved betingede branch-instruksjoner fordi vi ikke vet om hvi skal utføre hoppet før etter EX. Stalling er dårlig fordi vi mister 2 syklar på hver branch instruksjon. Predikerer at vi ikke trenger å hoppe. Feil: Må rtydde opp i IF, ID, EX. Flush: Setter kontrollsignalene til 0 i samlebåndet. Detektere forgreining tidlig: Kan legge til en ekstra ALU i ID som sjekker om registerverdiene er like, slik at man finner ut av det tidlig.		
T5.2: Unntak og avbrudd	• Studenten skal kunne forklare begrepene unntak og avbrudd, inkludert presise unntak. Unntak, exception: Ikke-forventet endring i kontrollfkyt. Eks. en ikke-implementert instruksjon. Deling på 0, ugyldig opcode, adresseringsfeil, overflow osv. Avbrudd, interrupt: Unntak der kilden er utenfor prosessoren, altså fra I/O. Tastatur, disk, timer (klokkeavbrudd). Presise unntak: For at operativsystemet skal håndtere unntaket må - Alle instruksjoner før være ferdige - Ingen instruksjoner etterpå har kjørt. Man kjører altså gjennom alle instruksjoner før unntaket og flusher alle instruksjoner etter unntaket. Gjør det muli å starte programmet på nytt fra der unntaket skjedde. • Studenten skal kunne beskrive hvordan presise unntak kan implementeres i 5-	• 4.30	• Ord. 22: Oppgave 10 (litt relevant)

stegssamlebåndet.



Lagrer adressen til instruksjonen i SEPC-register
 Lagrer grunnen til unntaket i SCAUSE-register
 Overfører kontroll til operativsystemet på en viss adresse
 Operativsystemet håndterer unntaket.

- Studenten skal kjenne til prinsippene bak hvordan presise unntak implementeres i prosessorer som utfører instruksjoner ut-av-rekkefølge.
- Reorder buffer, instruksjoner skrives tilbake til register i programrekkefølge
- Register renaming, fjerner WAR og WAW

ROB holder styr på rekkefølgen, register renaming holder styr på verdiene

T5.3:
Prossessorer
med høyere
ytelse

- Studenten skal kunne beskrive hvordan en prosessor utnytter parallellitet i tid og parallellitet i rom.
- Parallellitet i tid: samlebåndet**
Parallellitet i rom: Superskaleritet, et skritt jobber med flere instruksjoner samtidig.
 Krever mye ekstra programvare (eks ekstra ALU).

Fører til flere dataavhengigheter og farer

- 4.31
Ingen spesielt relevante

- Studenten skal kunne gjenkjenne og beskrive RAW, WAW og WAR farer og avhengighetene som skaper dem.

- **Raw - read after write:** Instruksjon B må lese en instruksjon som A først må skrive

add x3, x2, x1

sub x5, x3, x6

- **Waw - Write after write:** To instruksjoner skriver til samme register, og rekkefølgen må bevares.

add x3, x2, x1

sub x3, x5, x6

- **WAR - write after read:** Instruksjon B vil skrive til en registerplass som instruksjon A ikke har lest ennå

add x3, x2, x1

sub x2, x5, x6

HUSK at multiplikasjon og divisjon tar flere sykler!!

- Studenten skal forstå prinsippene bak hvordan «register renaming» fjerner WAW og WAR farer.

Register renaming: Legger til flere fysiske registre enn programmet kan se. Tilordner nye registre til instruksjonene slik at vi fjerner navneavhengigheter (WAW, WAR) og respekterer dataavhengigheter, RAW. Eks:

add a0, a1, a2 -> a0 -> p3

sub a0, a5, t0 -> a0 -> p7

Lager en tilordningstabell og oversetter gamle registernavn til nye.

- Studenten skal kjenne til fordeler og ulemper med «static multi-issue» prosessorer.
- Static multi-issue:** Kompilatoren avgjør hvilke instruksjoner som kan kjøres samtidig i samme klokkesyklus ved å finne uavhengige instruksjoner

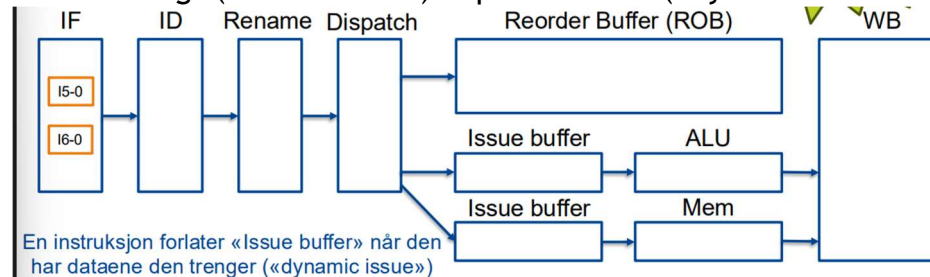
Fordeler:

- Enklere maskinvare
- Bedre ytelse, utnytter lokalitet i tid, mer ytelse per watt
- Kan redusere avhengigheter ved å flytte instruksjonene rundt

Ulemper:

- Svært avhengig av at kompilatoren oppdager avhengighetene og hva som kan kjøre i parallell. Må jobbe hardt.
- Mye parallellitet som kun oppstår i kjøretid (forgreiningsprediksjon og spekulativ minneaksess)

- Studenten skal kjenne til prinsippene bak hvordan prosessorer som utfører instruksjoner ut av rekkefølge («out of order») implementeres («dynamic multi-issue»).



Instruksjonene starter som normalt, går gjennom register renaming, og settes inn i en kø:

- Dispatch: holder instruksjoner til de er klare til å sendes videre til issue bufferene, bestemmer hvilket issue buffer de skal sendes til.
- Issue buffer: sender dataen videre til ALU eller Mem når vi har dataen/operandene vi trenger for å utføre handlingen og det neste steget er klart. Med 2 parallelle tråder kan vi hente data fra minnet og gjøre aritmetiske operasjoner samtidig.
- WB, write back, som vanlig
- Reorder buffer, ROB, lagrer rekkefølgen som instruksjonene kommer inn i. Slik vil

	instruksjonene gå ut av WB-registeret i samme rekkefølge som de kom inn i CPUen Rekkefølgen vil da være den samme til og med dispatch og ut av write back, men ut-av-rekkefølge i forgreiningene. • Studenten skal kunne forklare begrepet «speculation». Prosessoren utfører instruksjoner som den ikke er sikker på at den skal utføre	
arki og hurtigbuffer	• Studenten skal kunne forklare begrepene lokalitet i tid og lokalitet i rom og vite hvorfor lokalitet oppstår. Lokalitet i tid: Hvis et program bruker et dataelement, er det sannsynlig at det vil bruke det igjen snart. Lokalitet i rom: Hvis et program bruker et dataelement, er det sannsynlig at det snart bruker et element på en adresse i nærheten Flytter dataelementer nært prosessoren i raske minner når de aksesseres, og flytter dataelementer vi ikke har brukt på en stund. • Studenten skal forstå hvordan og hvorfor et hierarki av minner kan gi illusjonen av ett stort og raskt minne. Vi vil ha et stort og raskt minne. Mindre minner er typisk raskere enn store minner, men også mye dyrer og tar mer plass. Derfor: minnehierarki! Det vi bruker ofte, lagres i små, raske minner, det som brukes sjeldent lagres i trege, store minner. Da oppleves det som et stort og raskt minne. • Studenten skal forstå hvordan vi konstruerer et direktetilordnet hurtigbuffer (kunne gjengi og forklare figur 5.10) samt hvordan det kan utvides til å håndtere hurtigbufferblokker som består av mer enn ett ord.	• Kont 25: Oppgave 4 • Ord. 24: Oppgave 5 • Kont 24: Oppgave 4 • Ord. 23: Oppgave 6 • Ord. 22: Oppgave 8 • Kont 22: Oppgave 4 • Ord. 21: Oppgave 2 og 3 • Ord. 21: Oppgave 6

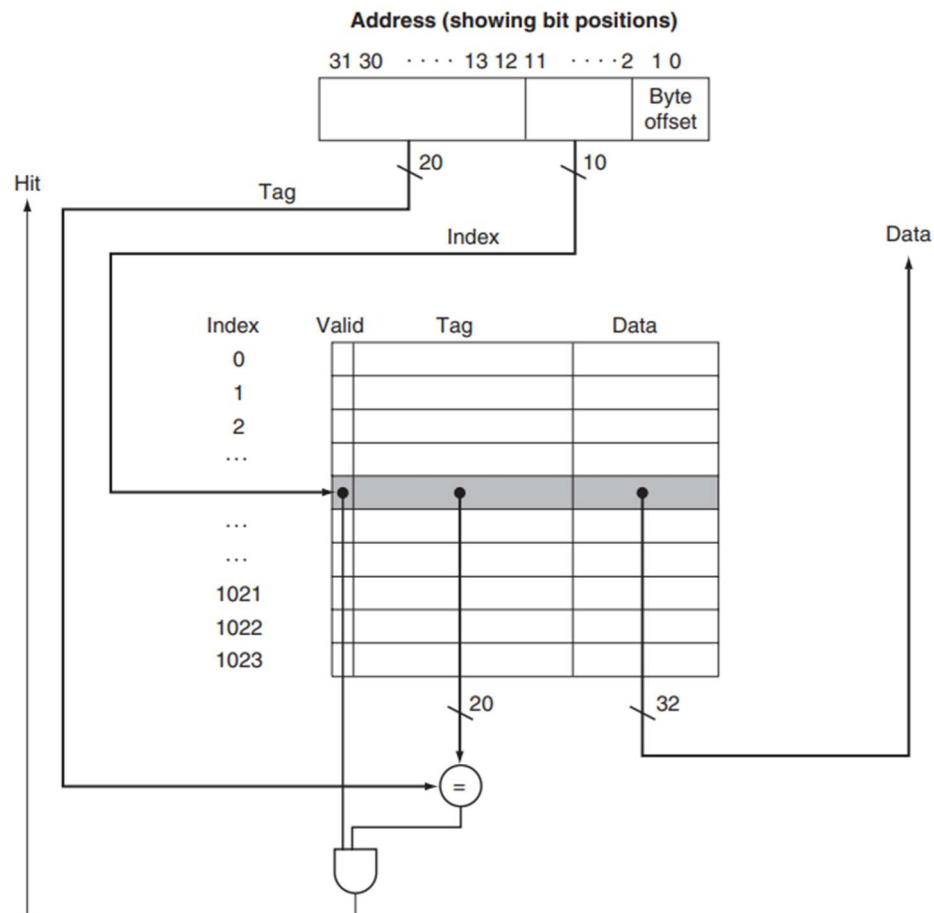


FIGURE 5.10 For this cache, the lower portion of the address is used to select a cache entry consisting of a data word and a tag. This cache holds 1024 words or 4 KiB. Unless noted otherwise, we assume 32-bit addresses in this chapter. The tag from the cache is compared against the upper portion of the address to determine whether the entry in the cache corresponds to the requested address. Because the cache has 2^{10} (or 1024) words and a block size of one word, 10 bits are used to index the cache, leaving $32 - 10 - 2 = 20$ bits to be compared against the tag. If the tag and upper 52 bits of the address are equal and the valid bit is on, then the request hits in the cache, and the word is supplied to the processor. Otherwise, a miss occurs.

Direktetilordnet: hver adresse kan kun ligge på 1 plass i cache.

Bruker samme adresse som i hovedminnet, men deler det opp og tolker det på en annen måte:

Offset: forteller hvilket ord i blokken som skal hentes (byte)

Index: bestemmer hvilken cache-entry / linje i cachen som adressa skal ligge i.

Tag: Identifiserer hvilken blokk fra minnet som ligger i denne cache-linjen

Valid: Eget for cache. 1 eller 0, sier om verdien er gyldig. Trenger disse ved oppstart og om vi skal tømme hurtigbufferet.

For å finne ut om vi har funnet riktig adresse, går vi til linja i cache med samme index som hovedminneadressa.

Så sammenlignes Tag for å sjekke om resten av minneadressa er korrekt.

Merk! Følge av dette er at for et f.eks. 2-veis set-assosiativ cache er det bare 2 minneadresser med samme index-bits som kan ligge i cachen. Dersom vi har 3 minneadresser som brukes ofte der index-bitene er de samme, kan bare 2 av de ligge i cache.

Utganger: hit og data

- Studenten kunne forklare hvordan hurtigbufferer kan integreres i en samlebåndsarkitektur. Kan ha instruksjonshurtigbuffer i IF og datahurtigbuffer i MEM. Treff er uproblematisk, bom (hit=0) må håndteres. Det enkleste er å fryse samlebåndet til vi har hentet den manglende hurtigbufferblokken fra hovedminnet.
- Studenten skal kjenne til prinsippene for hvordan hurtigbufferer håndterer skriveoperasjoner. Når vi skriver til hurtigbufferet må minnet oppdateres
 - **Write-through:** Oppdatere blokk hver gang minnet skrives
 - **Write back:** Oppdatere minnet når blokk kastes ut av hurtigbufferet. Må da vite om en hurtigbufferblokk har blitt skrevet til med:
dirty bit

Bom på skriveoperasjon:

- **No write allocate:** Skrive rett til minnet
- **Write allocate:** Henter blokk fra minnet før vi skriver til den

- Studenten skal kunne beregne hvordan minneaksesstid påvirker datamaskinens ytelse.

Husk å legge til tid brukt på å iterere gjennom de forskjellige registerne!

Minnelatens: ant klokkesykler det tar å hente en verdi fra minnet

$$\text{CPI} = C(\text{beregning}) / I(\text{total}) + C(\text{minne}) / I(\text{total})$$

Minne-/beregningsintensiv: et program er det hvis den bruker en stor andel av kjøretiden på minne/beregning

Average memory access latency: $C(\text{minne}) / \text{minneforespørsler}$

- Studenten skal kunne beregne hvordan treffraten i flernivå hurtigbuffer påvirker den gjennomsnittlige aksesstiden.

Samme prinsipp som over. Forventningsverdi

- Studenten skal forstå hvorfor settassosiative og fullassosiative hurtigbuffer kan øke treffraten og hvordan de konstrueres (gjengi og forklar figur 5.18).

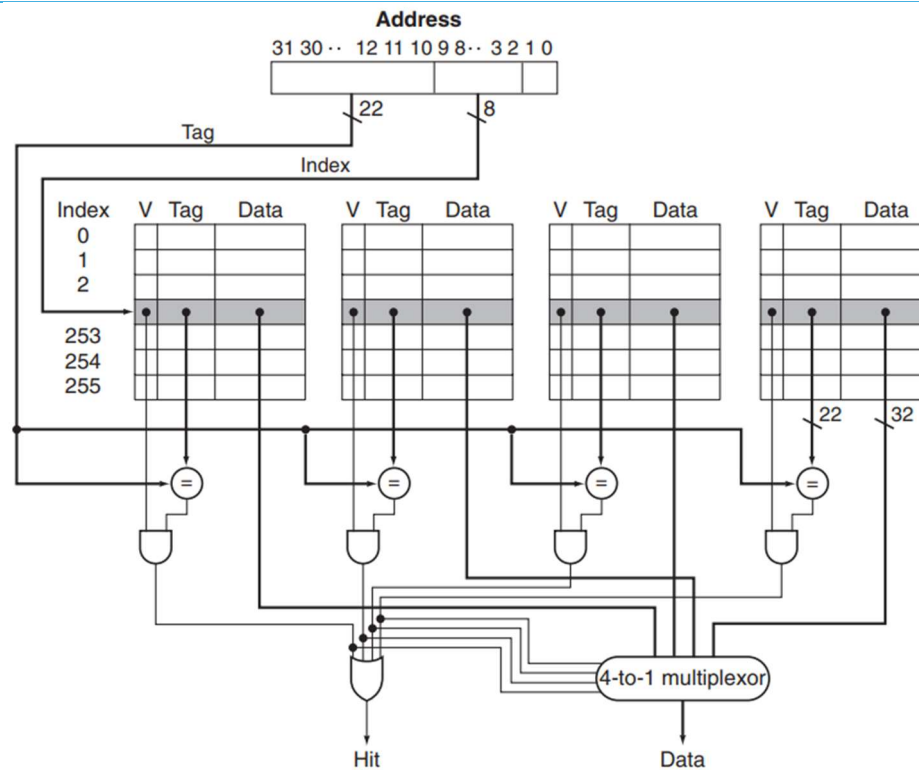


FIGURE 5.18 The implementation of a four-way set-associative cache requires four comparators and a 4-to-1 multiplexor. The comparators determine which element of the selected set (if any) matches the tag. The output of the comparators is used to select the data from one of the four blocks of the indexed set, using a multiplexor with a decoded select signal. In some implementations, the Output enable signals on the data portions of the cache RAMs can be used to select the entry in the set that drives the output. The Output enable signal comes from the comparators, causing the element that matches to drive the data outputs. This organization eliminates the need for the multiplexor.

Samme som figuren over, men her har vi 4-veis fullassosiativ hurtigbuffer. Slik kan flere minneadresser med samme index-bits lagres i hurtigbufferet. Sammenligner tag, treffer på en av dem, slik at riktig data kommer ut. Mer fleksibelt, kan plassere 1 adresse på flere steder. men dyrere fordi vi må sammenligne alle tags.

Set-assosiativ: Flere blokker i samme blokk. Da flyttes index mot venstre og vi får et felt med «block offset» som forteller hvilken blokk dataen skal ligge i.

Stor blokk: bedre utnyttelse av lokalitet i rom, men plass til færre blokker og økende kostnad ved å hente ut nye data

- Studenten skal kjenne til hvordan man på programvarenivå kan påvirke treffraten i hurtigbufferne.

Bruke lokalitet i tid og rom:

Lokalitet i tid:

God gjenbruk av variabler som allerede er i cache, eller strukturer.

Lokalitet i rom:

Data som ligger nært hverandre i minnet blir brukt samtidig

Fler-veis assosiativ cache gjør at flere adresser med samme index-bits kan være i bufferen, som vil øke treffraten. Gjør også at ulike adresser med samme index kaster ut hverandre.

Treffraten øker med størrelsen på hurtigbufferet, men aksessiden øker også

Få nivå hurtigbuffer!

Nivå 1: liten og rask

Nivå 2: større og ganske rask.

Øker treffraten fordi noen bom nå treffer i nivå-2-hurtigbufferet.

T6.2: Minne- teknologier

- Studenten skal kunne forklare begrepene volatilt og ikke- volatilt minne samt statisk og dynamisk minne.

Volatile minner: mister verdiene sine når vi skruer av strømmen

- SRAM
- DRAM

Ikke-volatile minner: Beholderr verdiene

- Flash
- Magnetisk harddisk

Statisk minne:

- Verdiene lagres i låser og beholdes så lenge strømmen er på

Dynamisk minne:

- Mister ladning, dermed også verdien, over tid.

- Studenten skal kjenne til omtrentlig aksestid og kostnad per bit for SRAM, DRAM, Flash, og magnetisk disk og kunne forklare hvordan dette påvirker hvordan teknologiene brukes i minnehierarkiet.

Memory technology	Typical access time	\$ per GiB in 2020
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$3–\$6
Flash semiconductor memory	5,000–50,000 ns	\$0.06–\$0.12
Magnetic disk	5,000,000–20,000,000 ns	\$0.01–\$0.02

- Studenten skal kunne konstruere et SRAM minne fra grunnleggende kombinatoriske og sekvensielle komponenter (kunne gjengi og forklare figur A.9.3).

Ingen
tilgjenge
lige
oppgaver

Ingen spesielt
relevante

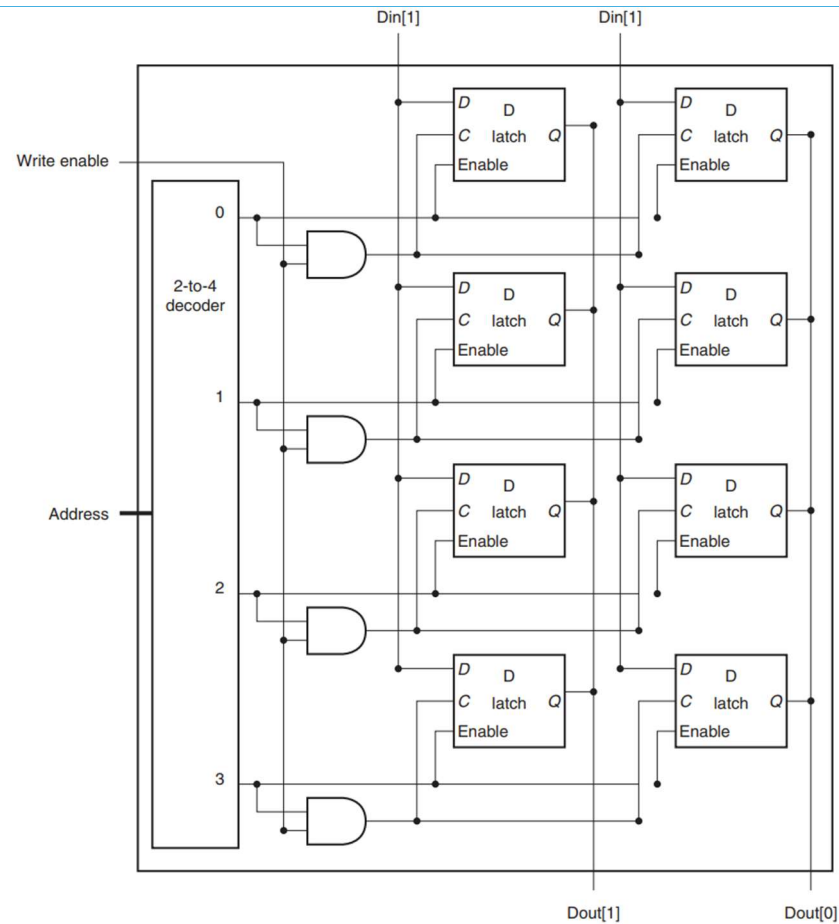


FIGURE A.9.3 The basic structure of a 4×2 SRAM consists of a decoder that selects which pair of cells to activate. The activated cells use a three-state output connected to the vertical bit lines that supply the requested data. The address that selects the cell is sent on one of a set of horizontal address lines, called word lines. For simplicity, the Output enable and Chip select signals have been omitted, but they could easily be added with a few AND gates.

Bruker tristate-buffer på Q-utgangen som enten har veldig høy utgangsimpedans eller kobler seg til utgangen. Slik kan man koble flere minneelementer til samme utgang.

Dersom write enable er høy, vil man kunne skrive til den valgte minneadressen (som også er høy) med dataen inn i D.

Denne har 2 bit ord og 4 adresser.

- Studenten skal kunne beskrive prinsippene for hvordan vi konstruerer DRAM minner (kunne gjengi og forklare figur A.9.5 og A.9.6).

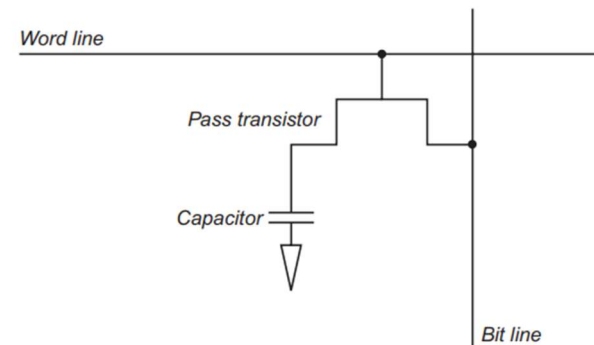


FIGURE A.9.5 A single-transistor DRAM cell contains a capacitor that stores the cell contents and a transistor used to access the cell.

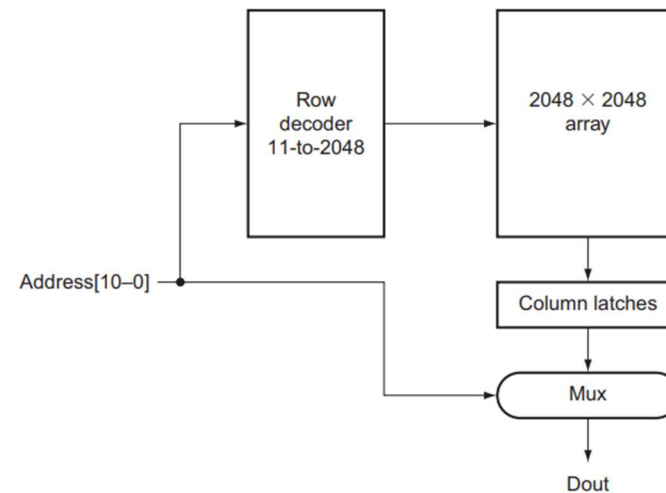


FIGURE A.9.6 A $4\text{M} \times 1$ DRAM is built with a 2048×2048 array. The row access uses 11 bits to select a row, which is then latched in 2048 1-bit latches. A multiplexor chooses the output bit from these 2048 latches. The RAS and CAS signals control whether the address lines are sent to the row decoder or column multiplexor.

	DRAM: En verdi er lagret i en kondensator og en transistor brukes til å få tilgang til den. Verdiene må skrives tilbake før de forsvinner. Sender inn 1 på word-line for å hente ut C-verdien i bit-line Adressen sendes i 2 steg: - Adressen til raden: henter ut en hel rad - Adressen til kolonnen: går inn i mux for å hente riktig verdi.		
T6.3: Virtuelt minne	• Studenten skal kunne forklare hvordan virtuelle adresser oversettes til fysiske adresser, inkludert arbeidsfordelingen mellom maskinvare og programvare. Det er uproblematisk og usikkert at datamaskinen kan skrive hvor som helst i minnet. Virtuelt minne: operativsystemet bestemmer over det fysiske minnet. En adresseoversetter oversetter de virtuelle adressene til de fysiske adressene, og motsatt. Slik får hvert program et isolert og sammenhengende adresserom. Operativsystemet deler minnet inn i sidetabeller og bruker sidetabellenen til å tilordne virtuelle sider til fysiske sider. Sidetabellen gir også info om hvilke prosesser som har lov til å skrive/lese denne siden. Sidetabellen er lagret i minnet og kan bare endres av operativsystemet. Blir en slags ordbok, men med info om tillatelser. • Studenten skal kunne forklare hvordan et «Translation Lookaside Buffer (TLB)» brukes til å implementere rask adresseoversettelse. Løser problemet med at sidetabellen er i minnet, men at oversettelsen må gjøres nær prosessoren for at det skal skje fort nok.	• 5.16 • 5.19	• Ord. 24: Oppgave 5.5 • Kont 23: Oppgave 5 og 6 • Ord. 21: Oppgave 4

TLB, hurtigbuffer for adresseoversettelser, translation lookaside buffer. Typisk 1 for instruksjoner og 1 for data, for å unngå strukturfare. Da kan man hente instruksjoner i IF og data i MEM i samlebånd, samtidig, parallellt.

Bom i TLB er dyrt, da må vi hente oversettelsen fra sidetabellen i minnet, legge den i TLBen, og deretter starte minneaskessen.

Optimalisering:

- Fullassosiativ TLB
- Maskinvarestøtte for å slå opp i sidetabell, reduserer kostnad ved bom.

Altså: ønsker å hente en virtuell adresse

- Slår opp i TLB
 - Får TLB hit = 1
 - Får ut den fysiske adressen og bruker den til å slå opp i hurtigbufferet, slik som i
- Studenten skal kunne forklare hvordan virtuelt minne kan brukes til å beskytte prosesser fra hverandres minneaksesser.
 - Brukermodus og systemmodus (modusbit): kan lese i brukermodus, men både lese og skrive i systemmodus. Slik kan ikke brukeren endre/oppdatere sidetabellene eller få tilgang til maskinvare den ikke skal, systemet gjør det. Sikrere.
 - Privilegerte instruksjoner: genererer unntak hvis de kjøres i brukermodus
 - Sidetabellpeker: peker til sidetabellen til prosessen som kjører akkurat nå (prosessene får hver sin sidetabell, se over)
 - TLB for å oversette raskt
 - Må kunne bytte mellom brukermodus og systemmodus

Kontekstbytte: bytte mellom prosesser. Operativsystemet lagrer tilstanden til nåværende prosess slik at den kan starte opp igjen der den slapp (PC, registerverdier, sidetabell osv). Typ funksjonskall for prosesser.
Operativsystemet er privilegert, kan gjøre ting som vanlige prosesser ikke kan.

- Studenten skal kunne forklare hvordan sidefeil håndteres.

Sidefeil: Når siden vi aksesserer ikke er i minnet

Eks: prosess a utfører load-instruksjon, men siden er ikke i sidetabellen:

- Prosessoren fullfører alle instruksjoner eldre enn load-instruksjonen og genererer et unntak (presise unntak)
- OS håndterer unntaket ved å:
Bytte til systemmodus og lagrer tilstanden til prosess a (PC, registerverdier, osv.)
Sette inn den manglende siden i minnet og oppdaterer sidetabellen
Legger inn den nye tilordningen i TLBen
Skriver tilbake tilstanden til prosess a
Bytter til brukermodus og returnerer kontroll til prosess a
- Nå funker det

Kan ta 1000vis av klokkesykler.

- Studenten skal kunne forklare hva en virtuell maskin er.

Flere operativsystemer som kjører på samme maskin via en, hypervisor/«virtual machine monitor», VMM. Slik kan hver prosess oppleve at de kjører alene på maskinen.

Eks. 2 operativsystemer lager 2 «grener» ut fra VMM som begge tror at de er alene og har hele maskinen for seg selv. vet ikke at den andre grenen eksisterer.

Grenene kan da kjøre parallellt og isolert på samme maskin. Brukes mye i skyer, hver kunde kan sette opp sitt programvaremiljø.

T7.1: Ytelse og Flynns taksonomi	• Studenten skal kunne forklare hvorfor det er enklere å utnytte parallellitet mellom uavhengige program enn innad i ett program. Uavhengige program kan kjøres uavhengig, altså er det ingen avhengigheter Innad i et program må vi passe på at alle avhengigheter respekteres når koden skrives. Vanskelig. • Studenten skal kunne bruke Amdahls lov til å analysere skalerbarheten til parallelle programmer. Parallellt program består av en sekvensiell del og parallell del. Øker ant prosessorer -> kjøretiden til den parallelle delen fordeles jevnt utover alle prosessorene, men kjøretiden til den sekvensielle delen er uendret. Parallell del: raskere Sekvensiell del: uendret	• 6.2 • 6.3 • 6.11	• Kont 25: Oppg. 5.1
---	--	--	--

- Studenten skal kjenne til konseptene sterk og svak skalering og kunne bruke dem til å vurdere skalerbarheten til ulike applikasjoner.

Sterk skalering: Vi antar at programmet gjør samme mengde arbeid selv om vi øker ant. prosessorer.

Dersom vi dobler ant. prosessorer, vil parallell kjøretid halveres.

Bra Når man vil gjøre samme oppgave raskere.

Svak skalering: Antar at problemstørrelsen vokser i takt med ant. prosessorer.

Hver prosessor gjør ca. samme mengde arbeid som før. Da holder forholdet mellom den sekvensielle og parallele delen seg omtrent konstant.

Bra når man vil øke problemstørrelsen og ressursene

Hvilken skalering som er best avhenger av situasjonen et program brukes i.

- Studenten skal kunne forklare Flynns taksonomi og kunne gi eksempler på SISD, SIMD, og MIMD maskiner.

Flynns taksonomi: Klassifisering av datamaskinarkitektur basert på ant. data- og instruksjonsstrømmer den kan håndtere samtidig. 4 klassifiseringer:

SISD, single instruction, single data. Eks: vanlig enkeltkjærne CPU, tradisjonell sekvensiell prosessor.

SIMD, en instruksjon utfører samme operasjon på flere datastrømmer samtidig. Eks: GPU. Stort sett alle høytytelsesprosessorer har SIMD-instruksjoner, for man får gjort mer arbeid per instruksjon. Muliggjør array-operasjoner.

MISD, Flere instruksjoner utføres samtidig på samme datastrøm. Uvanlig, kan brukes i spesialiserte systemer med feilsikring.

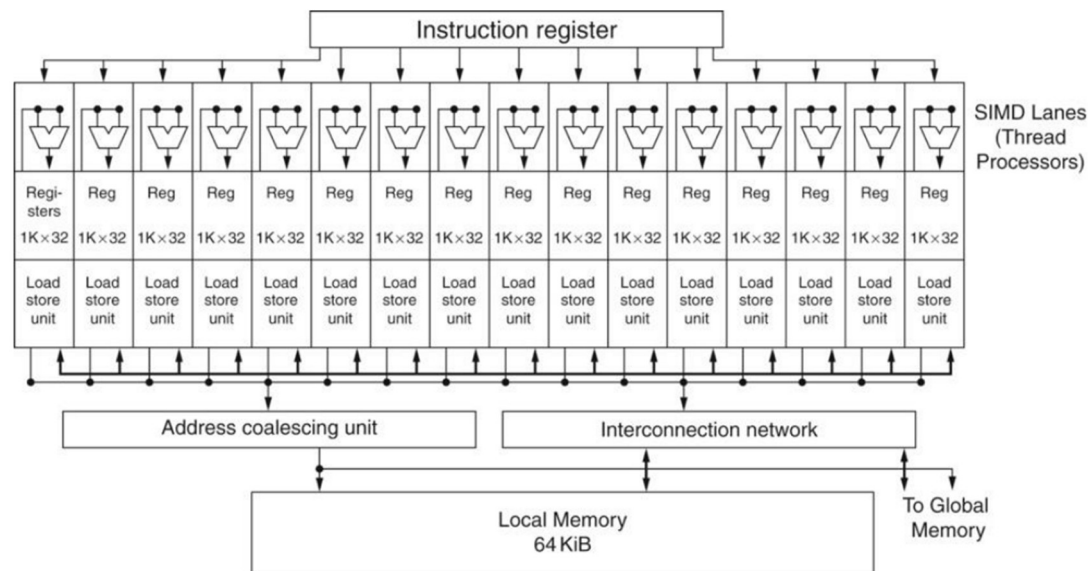
MIMD, flere instruksjoner på flere datastrømmer samtidig. Eks. flerkjerne-CPUer, datasentere

Kan koble sammen flere SIMD eller SISD for å få MIMD

	Studenten skal kunne bruke Roofline-modellen og konseptet operasjonsintensitet til å analysere hvorvidt en applikasjon er minnebundet eller beregningsbundet på en gitt datamaskin. Vi kan karakterisere applikasjoner basert på deres operasjonsintensitet: <i>(Ant operasjoner) / (byte hentet fra minnet)</i> Eks: Lav operasjonsintensitet: applikasjonen er mer minnebundet. Roofline-modellen: plotter operasjonsintensiteten til applikasjonen (x = ant. operasjoner, y = bytes hentet fra minnet) Kan bruke dette for å finne ut om ytelsen til en applikasjon er begrenset av beregning eller minnebåndbredde ved å se hvor grafen flater ut. Kan også bruke roofline-modellen til å analysere hvor godt en implementasjon utnytter maskinen.		
T7.2: Multiprosessorer	Studenten kan beskrive den overordnede arkitekturen til multiprosessorer med delt minne og distribuert minne samt kjenne til deres styrker og svakheter. Multiprosessor: flere prosessorer. Hver prosessor kan ofte ha flere kjernet. Delt minne: Alle prosessorene er koblet sammen til samme fysiske minnerom og kan lese og skrive til de samme minneadressene. De har egne cache Styrker: Enklere programmering, prosessorene får tilgang til felles data og god ytelse for programmer med hyppig datautveksling mellom prosessorene Svakheter: Skalering er vanskeligere, blir litt flaskehals når mange prosessorer skal ha tilgang til samme minne. K Distribuert minne: Alle prosessorene har sine egne fysiske minnerom og egne cache. Prosessorene kan likevel kommunisere via meldinger. Styrker: Kan skaleres godt til mange prosessorer, lokale minner gir rask tilgang, bra for storskala beregninger som kan deles opp i uavhengige oppgaver. Svakheter: Mer komplisert programmering, høy kommunikasjonskostnad hvis	- Forelesning 12 - Kapittel 6.4, 6.5 og 6.8 - Kapittel 5.10 - Kapittel 2.11	Kont 25: Oppg. 5.2

	prosessorene må utveksle mye info, mindre fleksibilitet • Studenten skal kunne beskrive den overordnede arkitekturen til en flerkjerneprosessor og kunne motivere for hvorfor arkitekturen er som den er. Hver prosessor har 2 egne nivåer hurtigbuffer, og så deler de siste nivået mellom seg. Delt minne. Dette utnytter prosessorens ressurser effektivt og gir god gjennomstrømning, throughput. Kan kjøre multithreads effektivt, kan lett kommunisere med hverandre og benytte seg av samme data. De er litt vanskeligere å skalere pga at det blir Trafikkpropp i delt minne og delt cache hvis mange kjerner må ha samme data samtidig. • Studenten skal kunne forklare hvorfor synkronisering er nødvendig og de grove trekkene i hvordan synkronisering implementeres. Nødvendig fordi noen ting må utføres i rekkefølge. Disse kalles kritiske seksjoner. Eks. når vi skal legge sammen delsummene fra hver tråd i et program. Må først lese og så skrive til en minnelokasjon samtidig som vi kan garantere for at ingenting har skjedd mellom lese- og skriveoperasjonen Load reserve og store conditional. Må kunne låse en ressurs, og deretter låse den opp igjen. • Studenten skal kjenne til behovet for å holde hurtigbufferne koherente og de grove trekkene i hvordan det løses. Hurtigbuffer-koherens: Alle kopier av samme minneadresse i ulike cacher viser samme verdi til enhver tid. Hvis 1 kjerne oppdaterer sin cache, må de andre kjernene se den oppdaterte verdien eller få indikasjon på at deres kopi er utdatert. Alle prosessorene må se operasjoner på samme dataelement i samme rekkefølge.		
--	--	--	--

	Cache coherence protocols: maskinvarestøtte som garanterer denne rekkefølgen. Så lenge alle er enige, går alt bra. • Studenten skal kunne beskrive minnekonsistensproblemet og kjenne til de grove trekkene i hvordan det løses. Konsistens: Samme som koherens, men på det globale hovedminnet. Oppstår når oppdateringer til delt minne fra én kjerne ikke nødvendigvis blir synlige umiddelbart for andre kjerner. DA kan tråder lese forskjellige verdier for samme variabel. Løsning: synkronisering med låser o.l., cache coherence protocols, og passe på at alle operasjoner ser ut til å skje i rekkefølge for alle tråder.		
T7.3 Akseleratorer (Grafikkprosessorer og domene-spesifikke akseleratorer)	• Studenten skal vite hva en heterogen datamaskin er og kunne forklare hvorfor datamaskiner med høy ytelse typisk er heterogene. Heterogene datamaskiner har beregningsenheter med ulike egenskaper. CPU for generelle beregninger, GPU er spesielt effektiv for grafikk, DSA domenespesifikke akseleratorer Med riktig enhet på rett applikasjon kan ytelsen bli veldig mye større fordi vi bruker de raskeste verktøyene for de riktige jobbene. Derfor er datamaskiner med høy ytelse typisk heterogene • Studenten skal kunne forklare den overordnede arkitekturen til grafikkprosessorer (GPUer) samt kunne beskrive programmeringsmodellen som benyttes når man bruker GPUer til generell beregning.	• Forelesning 12 • Kapittel 6.6, 6.7	• Kont 25: Oppgave 5.3 og 5.4 • Kont 24: Oppgave 5



GPU fokuserer på gjennomstrømning, ikke kjøretid.

Den fundamentale enheten er en tråd. Trådene samles i grupper («warps») som gjør samme operasjon på ulike data (SIMD).

Warps samles i thread blocks som er garantert til å kjøre på samme GPU-kjerne, streaming multiprocessor, SM. Disse kjøres parallellt.

Hver SM har tilgang til flere thread blocks til enhver tid. Hvis en av den stanser, bytter den til en annen thread block.

Har mange SMer som kjører parallellt. De har alle et delt cache-minne og et stort delt minne mellom alle Smer. Hver tråd har sine egne registre.

Kontrollenhet bestemmer hvilke warps som kjører, håndterer synkronisering og venting på minne.

Bruker altså mye mer parallellitet på programvarenivå for å skjule latens.

Programmering:

	Kernel: funksjon som kjøres 1 gang per datasett Hver tråd utfører kernel på ett element Alle trådene i en warp utfører samme instruksjon samtidig på ulike data Altså: skriver en kernel, som kjøres av mange tråder. Trådene grupperes i warps og igjen i thread blocks som kjører instruksjoner samtidig. Mange warps kan kjøre samtidig, men de kjører på samme kernel. • Studenten skal kunne beskrive likheter og forskjeller mellom GPUer, vektorprosessorer, og SIMD-instruksjoner. Likheter: - Utnytter parallellitet på data-nivå for å øke gjennomstrømning - GPU og vektorprosesser implementerer i praksis SIMD Ulikheter: - Arkitektur: GPU har masse små kjerner, vektorprosesser har en stor vektorenhet med store registre, det er få SIMD-enheter på en vanlig CPU - GPU kan ha 100000+ tråder, vektorer typ 64 og SIMD er mer begrenset - Minnehierarki: Register, delt og globalt minne for GPU, vektorregistre og vanlig cache for vektor og vanlige CPU-registere + cache for SIMD - Forskjellige instruksjoner: Kernels, tråder i warps. Vektorinstruksjoner. SIMD-instruksjoner.		
--	--	--	--

	• Studenten skal kunne forklare hva en domene-spesifikk akselerator (DSE) er og motivere for hvorfor disse kan oppnå høyere ytelse enn mer generelle arkitekturer (altså CPUer og GPUer). Enhet i prosessoren som er spesifikk for 1 type beregning / oppgave. Altså enda mer spesifikk og skreddersydd for 1 type operasjon enn CPU og GPU. Dette gir høyere effektivitet og ytelse for akkurat denne type arbeid. F.eks NPU, neural processing unit for AI-operasjoner. Ulempen er at vi må bygge den, og hvis man skal gjøre dette for alle «problemer», blir det veldig mange og veldig kostbart. Må også kanskje ha eget programmeringsspråk for denne.			
--	---	--	--	--